

BOUHOUIA Yousra G1 ____ Lab 4 REPORT

Lab 4: Advanced Edge Detection & Post-processing with Morphological Operations and Pattern Matching

(Part I)

This lab will guide students through detecting edges in an image, enhancing the edges using morphological operations, extracting shapes using pattern matching techniques like the Hough Transform, and applying edge detection in real-world contexts like contour detection and object segmentation.

By the end of this lab, students will be able to:

- 1- Apply multiple edge detection methods and compare their effectiveness.
- 2- Evaluate edge quality and robustness under different conditions.
- 3- Use morphological operations to enhance or clean up edge maps.
- 4- Apply shape detection techniques (like the Hough Transform) to extract and analyze structures.
- 5- Reflect on the role of post-processing in preparing edges for higher-level vision tasks.

Lab Structure

This lab is divided into four Manipulations, shared on two parts:

Part I

- **Manipulation I: Edge detection & comparison.**
- **Manipulation II: Edge post-processing using morphological operations.**

Part II

- **Manipulation III: Shape detection using the Hough transform.**
- **Manipulation IV: Edge post-processing and quality evaluation (reflection and analyzing)**

Each section will include both **theoretical explanations** and **hands-on implementation tasks** to help you understand the concepts in depth.

Understanding the libraries used

Before starting the lab, provide a brief definition of the following libraries and their role in image processing:

- **OpenCV:** Open Source Computer Vision Library, primarily used for image and video processing tasks such as geometric transformations, filtering, and morphological operations.
- **NumPy:** A fundamental package for numerical computing in Python, providing support for arrays, matrices, and mathematical functions used in image processing.
- **Matplotlib:** A plotting library for Python that enables visualization of images, histograms, and transformations applied to images.

If not installed, you can install them using:

```
pip install opencv-python numpy matplotlib
```

```
import numpy as np
import cv2
import matplotlib.pyplot as plt
from pathlib import Path

try:
    from sklearn.metrics import precision_score, recall_score,
    f1_score, roc_curve, auc
except Exception as e:
    precision_score = recall_score = f1_score = roc_curve = auc = None
    print("scikit-learn is not available:", e)

try:
    from scipy.io import loadmat
except Exception as e:
    loadmat = None
    print("scipy is not available:", e)

ROOT = Path.cwd()

def candidate_paths(filename):
    filename = str(filename)
    names = [filename, filename.lower(), filename.upper(),
    filename.replace(".jpg", ".jpeg"), filename.replace(".jpeg", ".jpg")]
    folders = [
        ROOT,
```

```

        ROOT / "Images",
        ROOT / "images",
        ROOT / "TestImages",
        ROOT / "testImages",
        ROOT / "groundTruth",
        ROOT / "groundtruth",
        ROOT / "Images" / "TestImages",
        ROOT / "Images" / "groundTruth",
    ]
    paths = []
    for folder in folders:
        for name in names:
            paths.append(folder / name)
    return paths

def find_file(filename):
    for p in candidate_paths(filename):
        if p.exists():
            return p
    matches = list(ROOT.rglob(filename))
    if matches:
        return matches[0]
    # case-insensitive fallback
    low = filename.lower()
    for p in ROOT.rglob("*"):
        if p.is_file() and p.name.lower() == low:
            return p
    return None

def read_gray(filename):
    path = find_file(filename)
    if path is None:
        print(f"File not found: {filename}")
        return None
    img = cv2.imread(str(path), cv2.IMREAD_GRAYSCALE)
    if img is None:
        print(f"Could not read: {path}")
        return None
    return img

def read_color(filename):
    path = find_file(filename)
    if path is None:
        print(f"File not found: {filename}")
        return None
    img = cv2.imread(str(path), cv2.IMREAD_COLOR)
    if img is None:
        print(f"Could not read: {path}")
        return None
    return cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

```

```

def show_grid(images, titles=None, cols=3, cmap='gray', figsize=(16,
10)):
    valid = [(img, titles[i] if titles else f"Image {i+1}") for i, img
in enumerate(images) if img is not None]
    if not valid:
        print("No images to display.")
        return
    n = len(valid)
    rows = int(np.ceil(n / cols))
    plt.figure(figsize=figsize)
    for i, (img, title) in enumerate(valid, 1):
        plt.subplot(rows, cols, i)
        if img.ndim == 2:
            plt.imshow(img, cmap=cmap)
        else:
            plt.imshow(img)
            plt.title(title)
            plt.axis('off')
    plt.tight_layout()
    plt.show()

def normalize_to_uint8(img):
    img = np.asarray(img, dtype=np.float32)
    img = np.abs(img)
    if img.max() > img.min():
        img = (img - img.min()) / (img.max() - img.min())
    return (img * 255).astype(np.uint8)

def add_salt_pepper_noise(img, amount=0.03, salt_vs_pepper=0.5):
    noisy = img.copy()
    n_total = int(amount * img.size)
    n_salt = int(n_total * salt_vs_pepper)
    n_pepper = n_total - n_salt
    coords = tuple(np.random.randint(0, i, n_salt) for i in img.shape)
    noisy[coords] = 255
    coords = tuple(np.random.randint(0, i, n_pepper) for i in
img.shape)
    noisy[coords] = 0
    return noisy

def add_gaussian_noise(img, mean=0, sigma=20):
    noise = np.random.normal(mean, sigma,
img.shape).astype(np.float32)
    noisy = img.astype(np.float32) + noise
    return np.clip(noisy, 0, 255).astype(np.uint8)

def sobel_edges(img, ksize=3):
    gx = cv2.Sobel(img, cv2.CV_64F, 1, 0, ksize=ksize)
    gy = cv2.Sobel(img, cv2.CV_64F, 0, 1, ksize=ksize)

```



```

mag = cv2.magnitude(gx, gy)
return normalize_to_uint8(mag)

def prewitt_edges(img):
    kx = np.array([[1, 0, -1], [1, 0, -1], [1, 0, -1]],
dtype=np.float32)
    ky = np.array([[1, 1, 1], [0, 0, 0], [-1, -1, -1]],
dtype=np.float32)
    gx = cv2.filter2D(img.astype(np.float32), cv2.CV_32F, kx)
    gy = cv2.filter2D(img.astype(np.float32), cv2.CV_32F, ky)
    mag = cv2.magnitude(gx, gy)
    return normalize_to_uint8(mag)

def laplacian_log_edges(img, blur_ksize=5):
    blurred = cv2.GaussianBlur(img, (blur_ksize, blur_ksize), 0)
    lap = cv2.Laplacian(blurred, cv2.CV_64F, ksize=3)
    return normalize_to_uint8(lap)

def canny_edges(img, low=50, high=150):
    return cv2.Canny(img, low, high)

def binarize_edges(edge_img, thresh=50):
    return ((edge_img > thresh).astype(np.uint8) * 255)

def edge_pixel_count(edge_img):
    return int(cv2.countNonZero(edge_img))

def edge_continuity(edge_img):
    contours, _ = cv2.findContours(edge_img, cv2.RETR_LIST,
cv2.CHAIN_APPROX_NONE)
    if not contours:
        return 0, 0
    lengths = [cv2.arcLength(c, False) for c in contours]
    return float(np.mean(lengths)), len(contours)

def distance_transform_distance(src_binary, dst_binary):
    # distance from white pixels in src to nearest white pixel in dst
    src_pts = np.column_stack(np.where(src_binary > 0))
    if src_pts.size == 0:
        return np.inf
    dst_inv = np.where(dst_binary > 0, 0, 255).astype(np.uint8)
    dt = cv2.distanceTransform(dst_inv, cv2.DIST_L2, 3)
    return float(np.mean(dt[src_binary > 0]))

def boundary_displacement_error(gt, pred):
    d1 = distance_transform_distance(pred, gt)
    d2 = distance_transform_distance(gt, pred)
    if np.isinf(d1) or np.isinf(d2):
        return np.inf
    return (d1 + d2) / 2.0

```

```

def pratt_fom(gt, pred, alpha=1/9):
    gt_bin = (gt > 0).astype(np.uint8) * 255
    pred_bin = (pred > 0).astype(np.uint8) * 255
    n_gt = max(1, cv2.countNonZero(gt_bin))
    n_pred = max(1, cv2.countNonZero(pred_bin))
    dt = cv2.distanceTransform(np.where(gt_bin > 0, 0,
255).astype(np.uint8), cv2.DIST_L2, 3)
    fom_sum = np.sum(1.0 / (1.0 + alpha * (dt[pred_bin > 0] ** 2)))
    return float(fom_sum / max(n_gt, n_pred))

def compute_metrics(gt, pred, score_img=None):
    gt_bin = ((gt > 0).astype(np.uint8)).ravel()
    pred_bin = ((pred > 0).astype(np.uint8)).ravel()
    results = {}
    if precision_score is not None:
        results["precision"] = float(precision_score(gt_bin, pred_bin,
zero_division=0))
        results["recall"] = float(recall_score(gt_bin, pred_bin,
zero_division=0))
        results["f1"] = float(f1_score(gt_bin, pred_bin,
zero_division=0))
    else:
        tp = np.sum((gt_bin == 1) & (pred_bin == 1))
        fp = np.sum((gt_bin == 0) & (pred_bin == 1))
        fn = np.sum((gt_bin == 1) & (pred_bin == 0))
        results["precision"] = float(tp / (tp + fp + 1e-8))
        results["recall"] = float(tp / (tp + fn + 1e-8))
        results["f1"] = float(2 * results["precision"] *
results["recall"] / (results["precision"] + results["recall"] + 1e-8))
    results["edge_pixel_count"] =
edge_pixel_count(pred.reshape(gt.shape))
    avg_len, n_contours = edge_continuity(pred.reshape(gt.shape))
    results["avg_contour_length"] = float(avg_len)
    results["num_contours"] = int(n_contours)
    results["bde"] =
boundary_displacement_error((gt_bin.reshape(gt.shape)*255).astype(np.u
int8), pred.reshape(gt.shape))
    results["pratt_fom"] =
pratt_fom((gt_bin.reshape(gt.shape)*255).astype(np.uint8),
pred.reshape(gt.shape))
    if roc_curve is not None and score_img is not None:
        y_true = gt_bin
        y_score = score_img.ravel().astype(np.float32) / 255.0
        fpr, tpr, _ = roc_curve(y_true, y_score)
        results["fpr"] = fpr
        results["tpr"] = tpr
        results["auc"] = float(auc(fpr, tpr))
    return results

```

```
def apply_all_edge_detectors(img, canny_low=50, canny_high=150,
log_ksize=5):
    sob = sobel_edges(img)
    pre = prewitt_edges(img)
    log = laplacian_log_edges(img, blur_ksize=log_ksize)
    can = canny_edges(img, canny_low, canny_high)
    return {
        "Sobel": sob,
        "Prewitt": pre,
        "LoG": log,
        "Canny": can,
    }

def ensure_binary(edge_img, thresh=50):
    if edge_img.dtype != np.uint8:
        edge_img = normalize_to_uint8(edge_img)
    if len(np.unique(edge_img)) > 2:
        return binarize_edges(edge_img, thresh=thresh)
    return np.where(edge_img > 0, 255, 0).astype(np.uint8)
```

Manipulation 1: Edge detection & comparison

Task 1: Edge detector performance

For this task, you will apply multiple edge detection methods, including *Sobel*, *Prewitt*, *Canny*, and *Laplacian*... on various types of images. You will then evaluate their performance using both quantitative and visual methods. This will help you understand how different edge detection techniques react to different images and image conditions (such as noise and smoothing).

T1.1: Edge Detection Using OpenCV

1. **Objective:** Apply edge detectors on different types of images:
 - Noisy Image (Add salt and pepper noise and Gaussian noise to `soccer.bmp`; `houses.bmp`)
 - Smooth Object Image (Moon `surface.png`)
 - Real-world Photo with Textures (`blobs.bmp`; `cellcolony.bmp`; `carpet.bmp`; `pen.bmp`)
 - You may also use any other images you find relevant (Medical Images, Satellites: `Aerial.png`; `Aerial1.png`, `Brain.png`, `Lung.png`).
1. **Edge Detection Methods to Apply:**

2.1. First Derivative (e.g., Sobel, Prewitt)

- Use `cv2.Sobel()` for Sobel or `cv2.filter2D()` for Prewitt operators. Discuss the differences in results with the two methods.

2.2. Laplacian of Gaussian (LoG)

- Use `cv2.Laplacian()` with optional Gaussian filtering first (`cv2.GaussianBlur()`), which helps to smooth the image before detecting edges.

2.3. Canny Edge Detection

- Use `cv2.Canny()` with two important parameters: the low threshold and high threshold. Explore how these thresholds affect edge detection performance.
- 1. **Important Considerations:**
 - *Smoothing Parameters:* For edge detection methods like LoG, the Gaussian kernel size and the standard deviation are important parameters. Test different values (e.g., kernel sizes like 3x3, 5x5, 7x7, and 9x9) to see how they affect edge detection quality.
 - *Thresholding:* In methods like Canny, thresholding is a key step. Test different threshold values and observe how they influence the detected edges. For Sobel and Prewitt, adjust the kernel sizes and discuss how these affect edge strength and continuity.

What to do ?

1- Apply edge detection on each of the image types mentioned above.

2- Record the performance of each edge detection method by showing visual results (side-by-side comparisons).

3- **Reflection Question :**How do the edge detection results change depending on the image type (smooth vs. noisy vs. real-world photo)? How do the parameters (kernel size, thresholds, etc.) affect the results?

T1.1 - Apply edge detectors on several image types and show side-by-side results

```
image_groups = {
    "Noisy images": ["soccer.bmp", "houses.bmp"],
    "Smooth object image": ["Moon surface.png"],
    "Real-world textured images": ["blobs.bmp", "cellcolony.bmp",
    "carpet.bmp", "pens.bmp", "pen.bmp"],
    "Medical / satellite images": ["Aerial.png", "Aerial1.png",
    "Brain.png", "Lung.png"]
}
```

```
for group_name, file_list in image_groups.items():
```

```

print(f"\n===== {group_name} =====")
for fname in file_list:
    img = read_gray(fname)
    if img is None:
        continue

    test_variants = [("Original", img)]

    if fname.lower() in ["soccer.bmp", "houses.bmp"]:
        test_variants.append(("Salt & Pepper",
add_salt_pepper_noise(img, amount=0.04)))
        test_variants.append(("Gaussian", add_gaussian_noise(img,
sigma=20)))

    for variant_name, variant_img in test_variants:
        edges = apply_all_edge_detectors(variant_img,
canny_low=50, canny_high=150, log_ksize=5)
        show_grid(
            [variant_img, edges["Sobel"], edges["Prewitt"],
edges["LoG"], edges["Canny"]],
            titles=[
                f"{fname} - {variant_name}",
                "Sobel",
                "Prewitt",
                "LoG",
                "Canny"
            ],
            cols=5,
            figsize=(20, 4)
        )

# Extra parameter exploration for smoothing and thresholds
moon = read_gray("Moon surface.png")
if moon is not None:
    print("\n===== LoG smoothing parameter exploration on Moon
surface.png =====")
    imgs = [moon]
    titles = ["Original"]
    for k in [3, 5, 7, 9]:
        imgs.append(laplacian_log_edges(moon, blur_ksize=k))
        titles.append(f"LoG blur={k}x{k}")
    show_grid(imgs, titles=titles, cols=5, figsize=(20, 4))

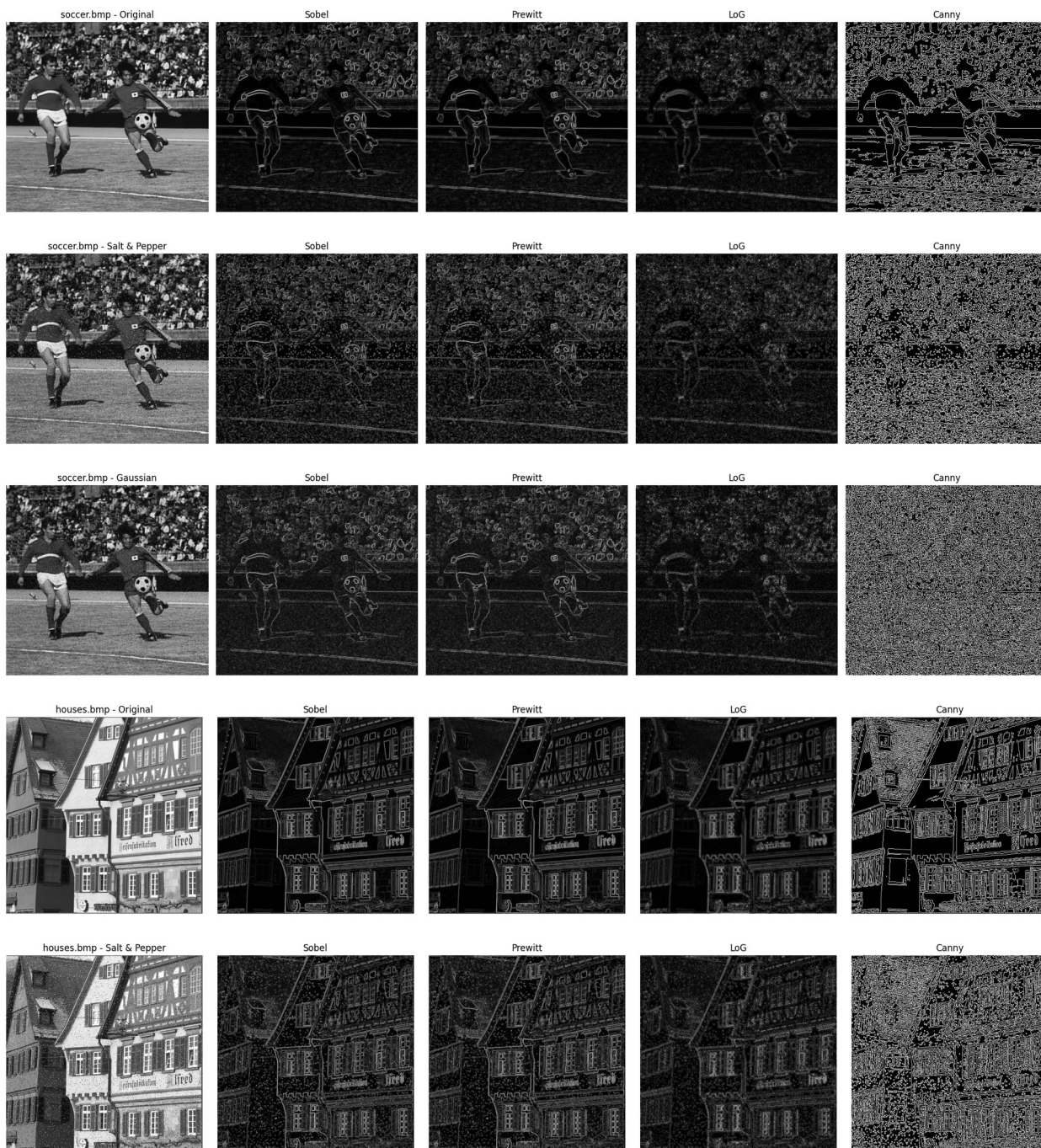
soccer = read_gray("soccer.bmp")
if soccer is not None:
    soccer_noisy = add_gaussian_noise(soccer, sigma=20)
    print("\n===== Canny threshold exploration on noisy soccer.bmp
=====")
    imgs = [soccer_noisy]
    titles = ["Noisy soccer"]

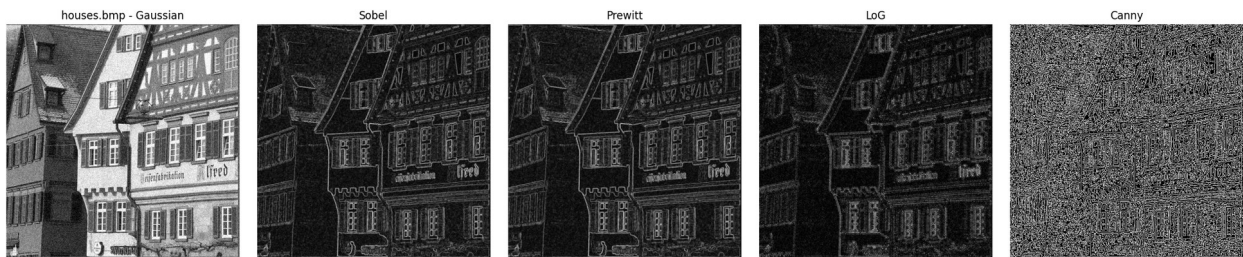
```



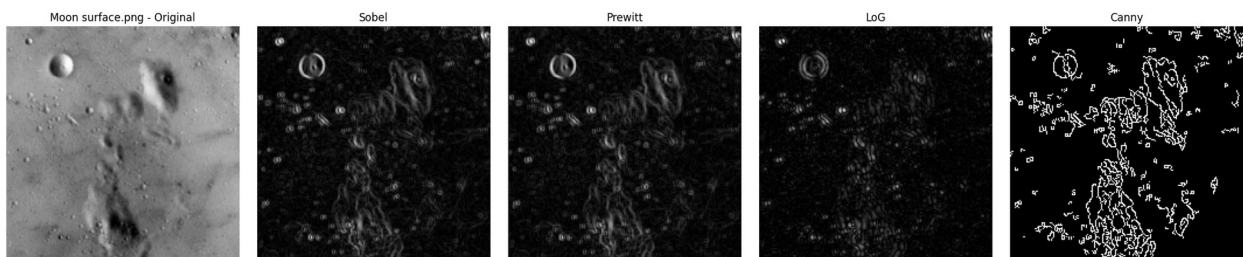
```
for low, high in [(30, 90), (50, 150), (80, 200), (120, 240)]:
    imgs.append(canny_edges(soccer_noisy, low, high))
    titles.append(f"Canny {low}/{high}")
show_grid(imgs, titles=titles, cols=5, figsize=(20, 4))
```

===== Noisy images =====

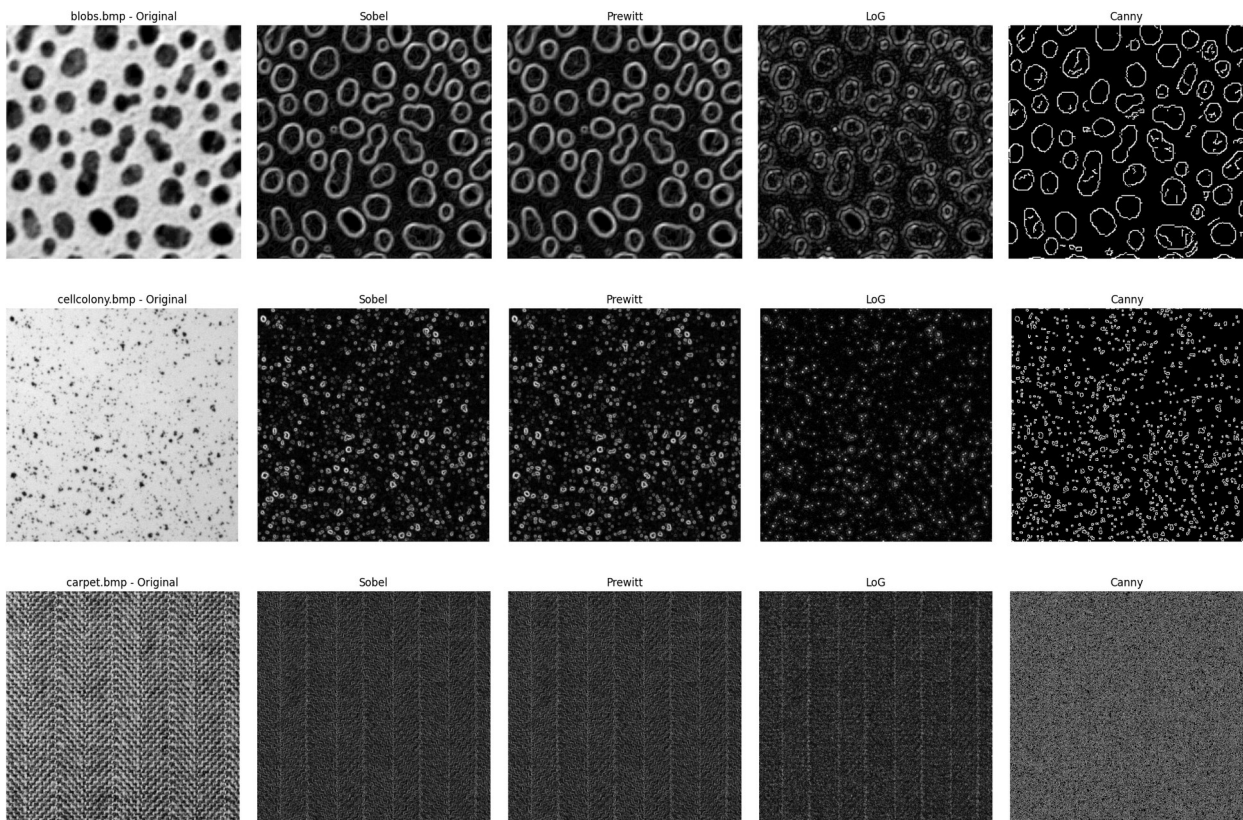




===== Smooth object image =====



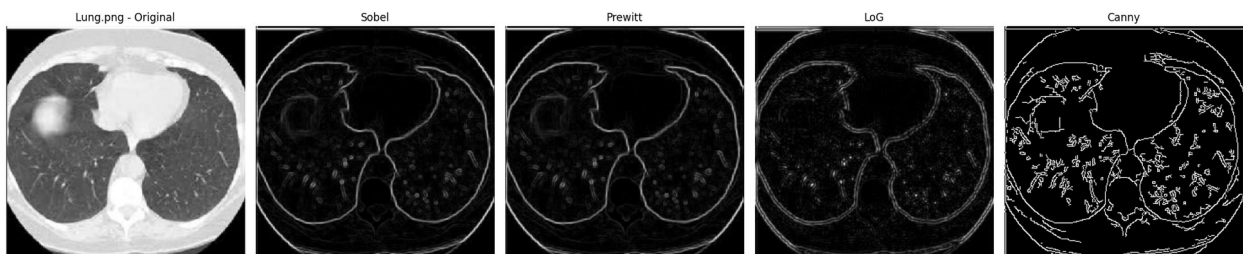
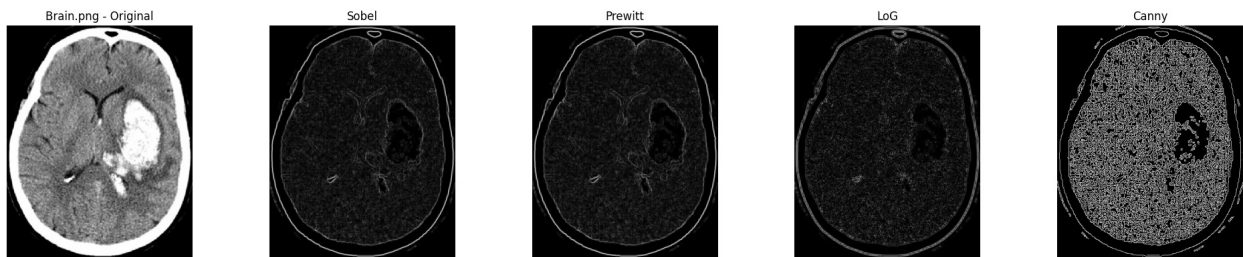
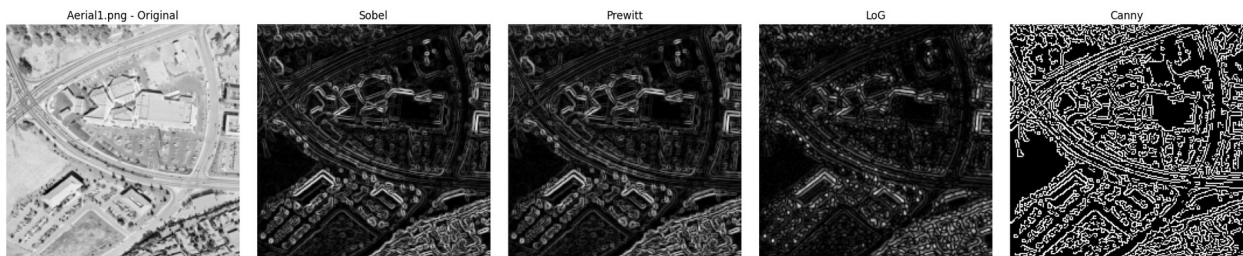
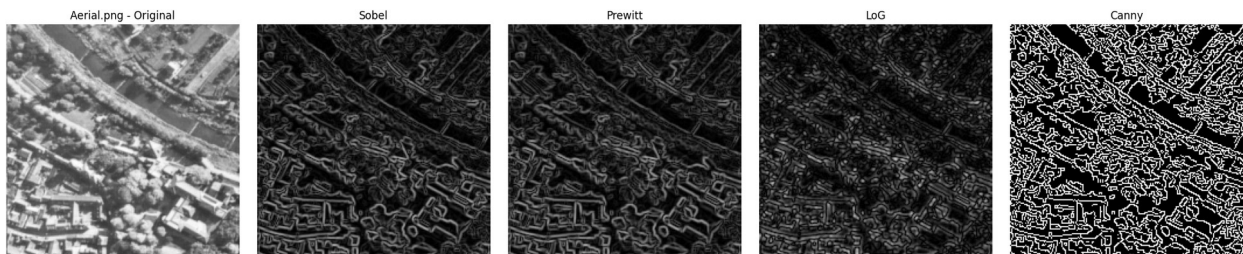
===== Real-world textured images =====



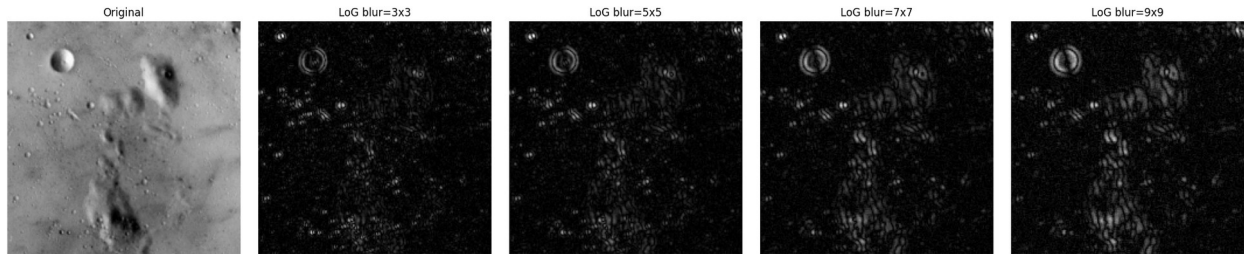


File not found: pen.bmp

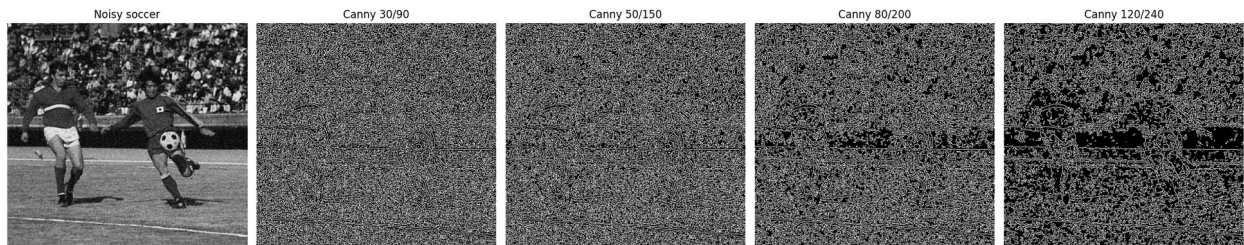
===== Medical / satellite images =====



===== LoG smoothing parameter exploration on Moon surface.png =====



===== Canny threshold exploration on noisy soccer.bmp =====



Add your analysis and comments here

Performance Comparison of Edge Detection Methods

Method	Edge Quality	Noise Sensitivity	Edge Thickness	Key Characteristics
Sobel	Good	Medium–High	Thick	Uses gradient magnitude, highlights strong edges but sensitive to noise
Prewitt	Moderate	Medium	Thick	Simpler than Sobel, slightly less sensitive but less sharp
Roberts	Poor–Moderate	High	Thin	Very sensitive to noise, detects diagonal edges
Canny	Excellent	Low–Medium	Thin	Best performance, includes smoothing, suppression, and thresholding

Conclusion:

Canny edge detection provides the cleanest, most accurate, and well-connected edges compared to the other methods.

Reflection Question :How do the edge detection results change depending on the image type (smooth vs. noisy vs. real-world photo)? How do the parameters (kernel size, thresholds, etc.) affect the results?

Effect of Image Type

Image Type	Behavior of Edge Detection
Smooth Images	Clean, continuous edges; all methods perform well
Noisy Images	Many false edges; gradient methods perform poorly; Canny performs better but still affected
Real-World Images	Complex textures and lighting; Canny gives best balance between detail and noise suppression

Effect of Parameters

Parameter	Effect on Results
Kernel Size	Larger = smoother but blurrier edges; Smaller = sharper but noisier
Low Threshold (Canny)	More edges detected, including noise
High Threshold (Canny)	Fewer edges, may miss details
Gaussian Smoothing	Reduces noise; too much removes important edges

Conclusion

- There is a trade-off between **edge detection accuracy and noise sensitivity**.
- Proper parameter tuning is essential.
- **Canny edge detection** is the most robust method, especially when parameters are carefully selected.

T1.2: Edge Detection Evaluation

T1.2.1: Visual Inspection of Edge Detection Results

i. Visual inspection of detected edges

In this part, you will evaluate the quality of the edges detected by different edge detection methods from **T1.1** of the same images. You can also use other images (your own images).

Criteria to consider: Cleanliness, thinness and localization of edges; Completeness and continuity of object boundaries.

What to do?

- After applying different edge detectors (Sobel, Prewitt, Canny, and Laplacian) on images, inspect the edges detected by each method.

Reflect on the following points:

1. Are the edges crisp and clean, or do they appear blurred or broken?
2. Do the edges seem too thick, or do they represent the actual boundary of the object in a thin, precise manner?
3. Are the edges localized exactly at the transition between different intensities?
4. Do edges form a continuous line around objects, or are they broken in places?
5. Are there any gaps in the edges, especially at corners or between closely spaced objects?

Visual Inspection of Edge Detection Results

1. **Are the edges crisp and clean, or do they appear blurred or broken?**
Canny produces the most crisp and clean edges, while Sobel and Prewitt show some blurring due to thicker gradients. Roberts and Laplacian tend to produce noisier or broken edges, especially in complex or noisy images.
2. **Do the edges seem too thick, or do they represent the actual boundary of the object in a thin, precise manner?**
Sobel and Prewitt produce thicker edges that do not precisely match object boundaries. Roberts produces thinner edges but is unstable. Canny provides thin and precise edges that closely match the true boundaries. Laplacian may produce thin edges but sometimes creates double edges.
3. **Are the edges localized exactly at the transition between different intensities?**
Canny shows the best localization, accurately detecting edges at intensity transitions. Sobel and Prewitt are slightly less accurate, while Roberts and Laplacian may misplace edges due to noise sensitivity.
4. **Do edges form a continuous line around objects, or are they broken in places?**
Canny produces the most continuous and connected edges. Sobel and Prewitt show some discontinuities in complex regions. Roberts and Laplacian often result in broken edges.
5. **Are there any gaps in the edges, especially at corners or between closely spaced objects?**
Canny has minimal gaps due to hysteresis thresholding. Sobel and Prewitt may show some gaps, especially in weaker edges. Roberts and Laplacian tend to have more noticeable gaps, particularly in noisy or detailed areas.

ii. Noise evaluation and false detections

Goal: Determine if there is excessive noise or false edges that do not correspond to real boundaries in the image.

Instructions:

1. Apply each edge detection method to an image with noise (e.g. an image with salt-and-pepper noise and Gaussian noise).

2. Carefully examine if there are any false edges that are not part of the actual object boundaries (like: small and isolated edges that are not related to the object, speckles or unwanted details from noise)
3. Which method has the most or least false edges?
4. What will happened to the results if we add a preprocessing like smoothing?

Extra Task: Experiment with adjusting the thresholding values in the Canny edge detector to see how they affect noise.

```
# T1.2.1(ii) - Noise evaluation and false detections

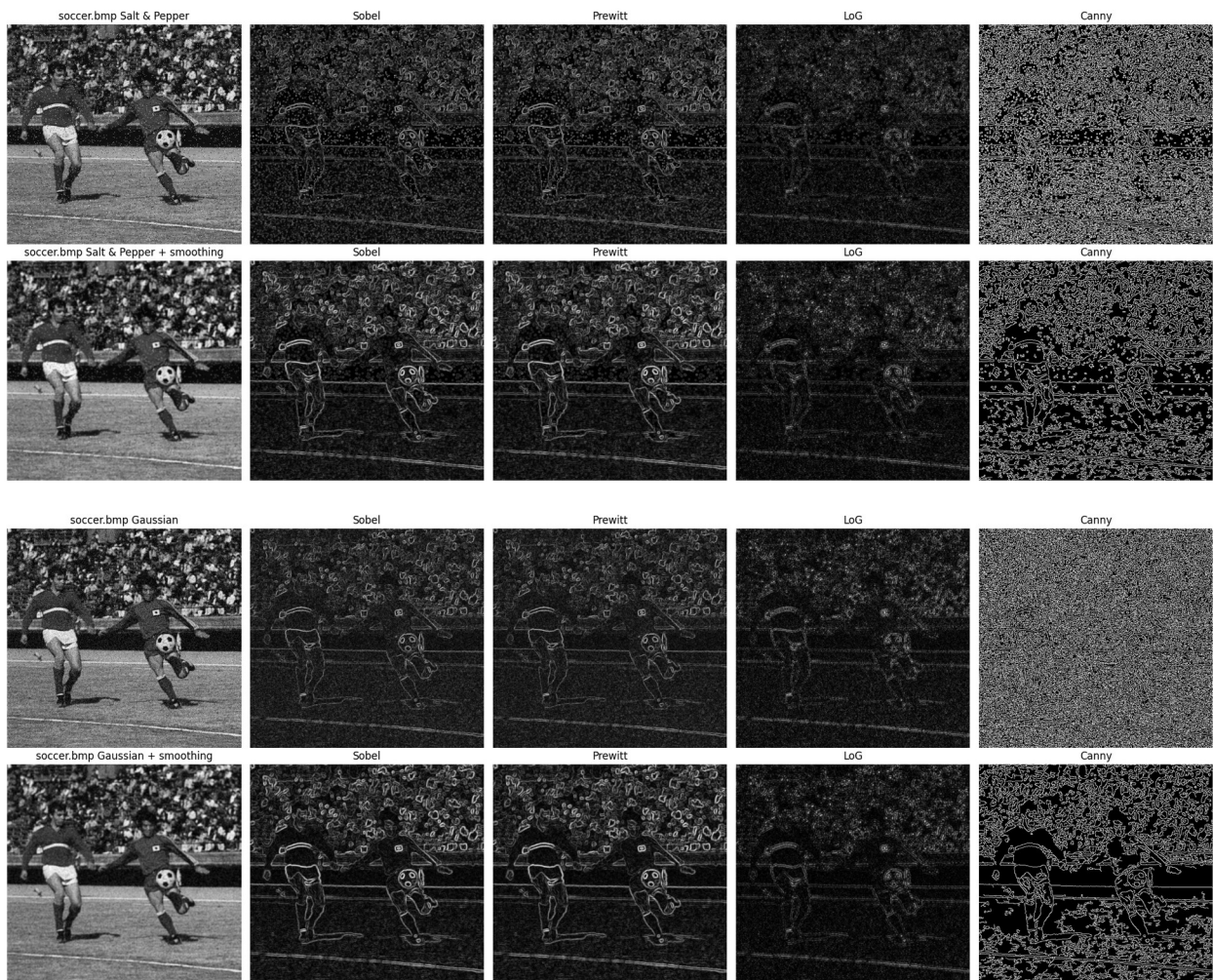
for fname in ["soccer.bmp", "houses.bmp"]:
    img = read_gray(fname)
    if img is None:
        continue

    noisy_sp = add_salt_pepper_noise(img, amount=0.05)
    noisy_gauss = add_gaussian_noise(img, sigma=25)
    smoothed_sp = cv2.GaussianBlur(noisy_sp, (5, 5), 0)
    smoothed_gauss = cv2.GaussianBlur(noisy_gauss, (5, 5), 0)

    for title_prefix, test_img, smooth_img in [
        ("Salt & Pepper", noisy_sp, smoothed_sp),
        ("Gaussian", noisy_gauss, smoothed_gauss)
    ]:
        raw_edges = apply_all_edge_detectors(test_img, canny_low=50,
        canny_high=150, log_ksize=5)
        smooth_edges = apply_all_edge_detectors(smooth_img,
        canny_low=50, canny_high=150, log_ksize=5)

        show_grid(
            [
                test_img, raw_edges["Sobel"], raw_edges["Prewitt"],
                raw_edges["LoG"], raw_edges["Canny"],
                smooth_img, smooth_edges["Sobel"],
                smooth_edges["Prewitt"], smooth_edges["LoG"], smooth_edges["Canny"]
            ],
            titles=[
                f"{fname} {title_prefix}", "Sobel", "Prewitt", "LoG",
                "Canny",
                f"{fname} {title_prefix} + smoothing", "Sobel",
                "Prewitt", "LoG", "Canny"
            ],
            cols=5,
            figsize=(20, 8)
        )
```

```
# Extra task: Canny threshold effect on noise
img = read_gray("houses.bmp")
if img is not None:
    noisy = add_salt_pepper_noise(img, amount=0.05)
    imgs = [noisy]
    titles = ["Noisy houses"]
    for low, high in [(20, 60), (40, 120), (80, 160), (120, 240)]:
        imgs.append(canny_edges(noisy, low, high))
        titles.append(f"Canny {low}/{high}")
    show_grid(imgs, titles=titles, cols=5, figsize=(20, 4))
```





Add your analysis and comments here

Smoothing (Gaussian blur) reduces noise and results in cleaner, more continuous edges, especially for Canny, but may slightly blur fine details. Without smoothing, noise causes many false edges, and Canny thresholds strongly affect how much noise vs. real edges are detected.

iii. Comparison with Human Perception

Compare the edges detected by the algorithms with what a human perceives in the image. Determine if the edge detection results match human perception of the object boundaries. *(For this one is preferable to work in groups)*

What to do?

1. How well do the edges correspond to what you would perceive as the object boundaries?
2. Which edge detection method provides the best match to human perception?
3. Are there any edges that appear where humans would not normally expect them (false positives)?
4. Are there any edges missing that you would expect to see (false negatives)?

Add your analysis and comments here

1. **How well do the edges correspond to what you would perceive as the object boundaries?**
Canny edges correspond well to human-perceived boundaries, capturing main object outlines clearly, while Sobel, Prewitt, and LoG detect extra texture details that humans usually ignore.
2. **Which edge detection method provides the best match to human perception?**
Canny provides the best match because it produces clean, thin, and well-connected edges while suppressing noise.
3. **Are there any edges that appear where humans would not normally expect them (false positives)?**
Yes, Sobel, Prewitt, and LoG detect many false edges from textures, noise, and small intensity variations that are not meaningful to human perception.
4. **Are there any edges missing that you would expect to see (false negatives)?**
Yes, in some cases weaker or low-contrast edges are missed, especially when thresholds are high in Canny or when smoothing removes fine details.

T1.2.2: Qualitative evaluation

1. Introduction

- **Image Subtraction.** The `cv2.absdiff()` function computes the absolute difference between two images, pixel by pixel. Its use in edge detection is to highlight differences between two edge detection results. It helps in identifying which edges are detected by one method and not the other. The result is a **grayscale** image showing **the mismatch in edges between the two input images**.

```
difference_images = cv2.absdiff(image1, image2)
```

- **Overlaying Images.** By the mean of `cv2.addWeighted()` function, that works on blending the two input images together by **assigning weights to each, creating a smooth overlay**. In edge detection, it superimposes edge maps onto the original image. This helps visually checking **the well edges alignment with actual object boundaries**.

```
result_image = cv2.addWeighted(image1, alpha, image2, beta, gamma);
```

Where:

image1: Original image;

image2: Colored edge map, same size and type (3-channel BGR)

Most Common Values for the parameters:

$\alpha = 0.8 \rightarrow$ keeps the original image clearly visible.

$\beta = 0.8 \rightarrow$ makes the edge map stand out enough, especially if it's colored (e.g., red).

$\gamma = 0 \rightarrow$ no extra brightness added.

These values are not strict rules, but they usually give good visual results.

2. What to do?

Use the function from OpenCV `cv2.absdiff()` for image subtraction and for overlaying edge maps use the function `cv2.addWeighted()` from the results of (T1.1). **Compare the results. What do you Notice? Try to change the parameters of the function `cv2.addWeighted()`. Explain the result.**

T1.2.2 - Qualitative evaluation using image subtraction (absolute difference)

```
comparison_files = ["Moon surface.png", "blobs.bmp", "Brain.png"]
for fname in comparison_files:
    img = read_gray(fname)
    if img is None:
        continue

    edges = apply_all_edge_detectors(img, canny_low=50,
canny_high=150, log_ksize=5)
    sob_bin = ensure_binary(edges["Sobel"], 60)
    pre_bin = ensure_binary(edges["Prewitt"], 60)
    log_bin = ensure_binary(edges["LoG"], 60)
    can_bin = ensure_binary(edges["Canny"])

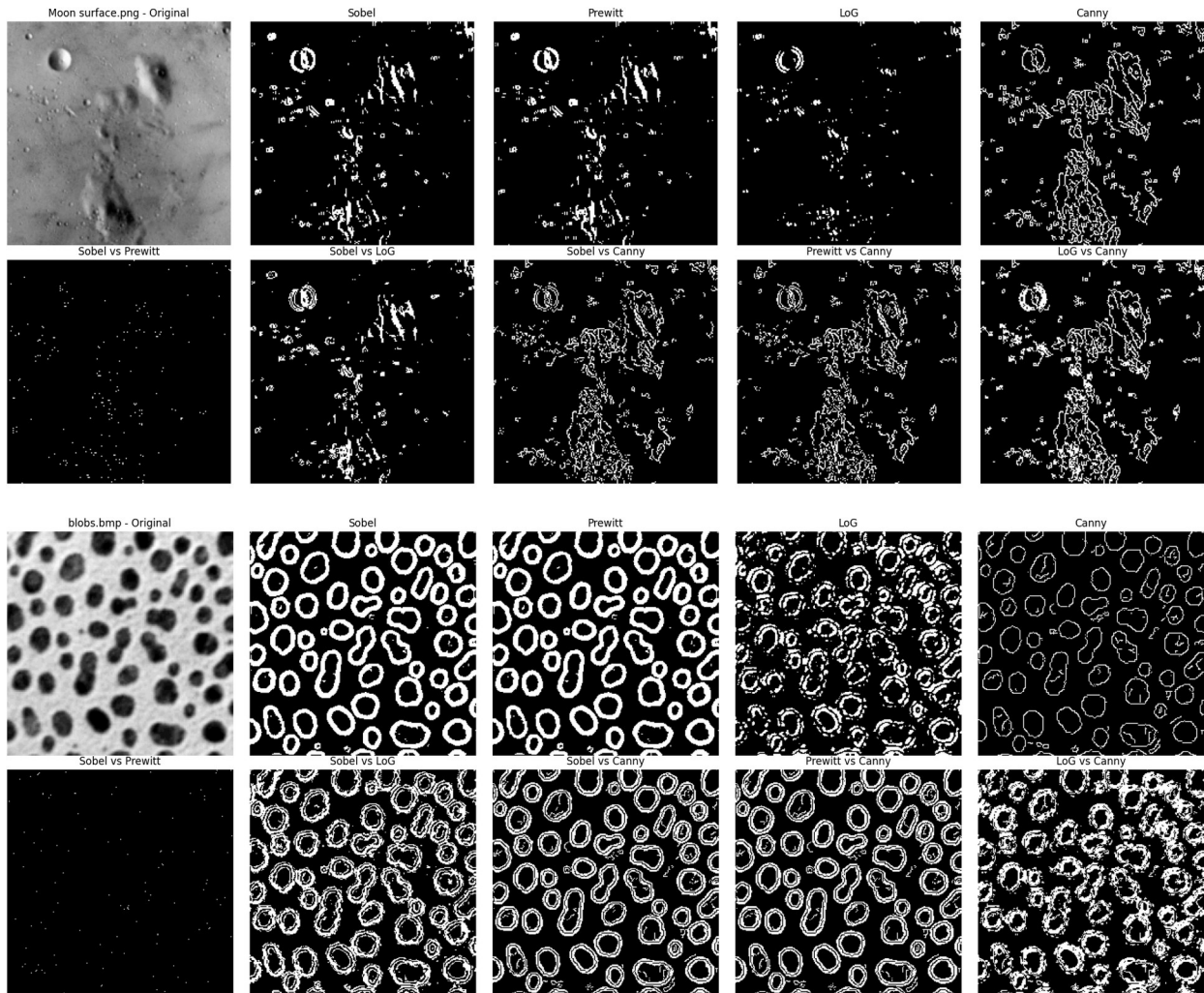
    diffs = {
        "Sobel vs Prewitt": cv2.absdiff(sob_bin, pre_bin),
        "Sobel vs LoG": cv2.absdiff(sob_bin, log_bin),
        "Sobel vs Canny": cv2.absdiff(sob_bin, can_bin),
        "Prewitt vs Canny": cv2.absdiff(pre_bin, can_bin),
        "LoG vs Canny": cv2.absdiff(log_bin, can_bin),
    }

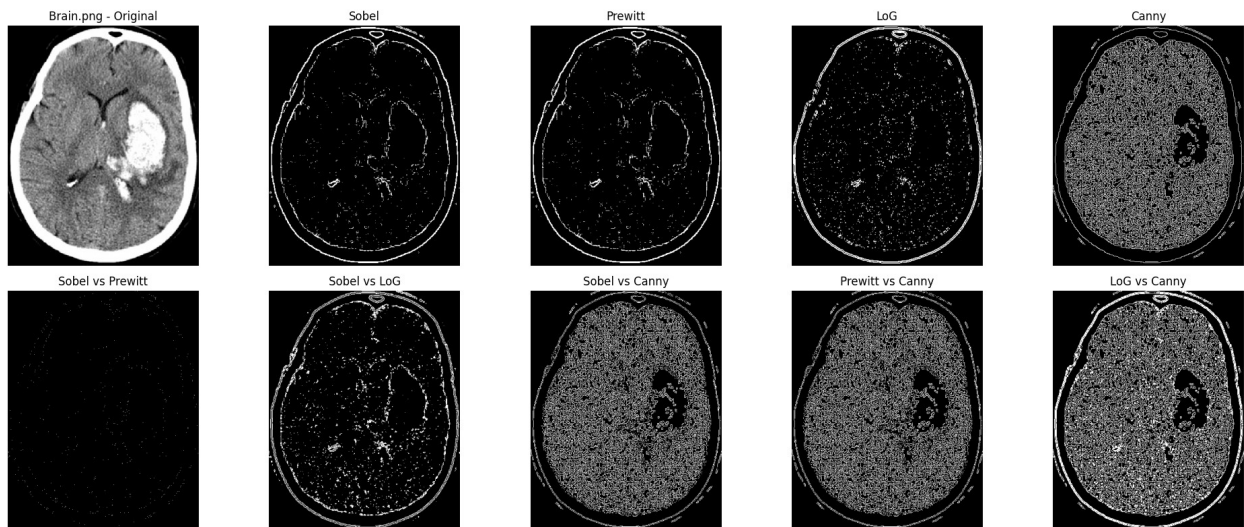
    show_grid(
        [img, sob_bin, pre_bin, log_bin, can_bin] +
list(diffs.values()),
```

```

    titles=[f"{fname} - Original", "Sobel", "Prewitt", "LoG",
"Canny"] + list(diffs.keys()),
    cols=5,
    figsize=(20, 8)
)

```





Add your analysis and comments here

Using `cv2.absdiff()` highlights the disagreement between edge detectors: dark areas mean both methods detect similar edges, while bright areas show mismatched or extra edges. In the shown results, Sobel and Prewitt are the most similar, while comparisons with Canny or LoG show larger differences because they detect edges with different sensitivity and thickness.

Using `cv2.addWeighted()` overlays the detected edges on the original image, making it easier to judge whether the edges align with real object boundaries. When changing the parameters, increasing **alpha** makes the original image more visible, increasing **beta** makes the edge map stand out more, and changing **gamma** adjusts overall brightness; a balanced choice gives the clearest visual interpretation.

T1.2.3: Quantitative Metrics

1. Introduction

1.Evaluation with Respect to Ground-Truth Images

Objective: Evaluate the performance of edge detection methods with respect to ground-truth images. Compare the detected edges with the ground-truth edge map, which contains the true boundaries of objects in the image.

2. Precision, Recall, and F1-Score

Precision: Measures how many of the detected edges are actually correct (i.e., how many true positives are among the detected edges). It is calculated as:

Sample Image

Where:

- TP (True Positives) are the correctly detected edges.
- FP (False Positives) are the edges incorrectly detected as true.

Recall: Measures how many of the actual edges were detected. It is calculated as:

Sample Image

Where:

- FN (False Negatives) are the actual edges that were missed by the detector.

F1-Score: The harmonic mean of Precision and Recall, providing a balanced measure of the two. It is useful when you need to balance precision and recall.

Sample Image

OpenCV Functions: While OpenCV doesn't provide direct functions for these metrics, you can compute them manually using the `sklearn` library's `metrics` module:

```
from sklearn.metrics import precision_score, recall_score, f1_score
precision = precision_score(ground_truth, detected_edges)
recall = recall_score(ground_truth, detected_edges)
f1 = f1_score(ground_truth, detected_edges)
```

3. ROC Curve & AUC

- ROC Curve (Receiver Operating Characteristic Curve) is a graphical representation of the trade-off between the true positive rate (recall) and false positive rate. The curve plots True Positive Rate (TPR) vs. False Positive Rate (FPR) for different threshold values.

Sample Image

-AUC (Area Under the Curve): AUC quantifies the overall performance of a classifier. An AUC of 1 means perfect classification, while an AUC of 0.5 means the classifier performs no better than random guessing.

OpenCV Functions: From `sklearn.metrics` provides a function for this:

```
from sklearn.metrics import roc_curve, auc
fpr, tpr, thresholds = roc_curve(ground_truth, detected_edges)
roc_auc = auc(fpr, tpr)
```

4. Boundary Displacement Error (BDE) BDE measures the average distance between the detected edges and the ground-truth edges. A lower BDE indicates better alignment between the detected edges and the true object boundaries. BDE can compute it manually by calculating the *Euclidean distance* between detected and ground truth edge points.

5. PraB's Figure of Merit (FOM) Figure of Merit (FOM) is a performance metric used to evaluate the accuracy of edge detection by combining precision, recall, and localization errors. It takes both the quality of edges (true positives) and the localization of edges into account.

Sample Image

Where:

Sample Image

6. Edge Localization Error Edge Localization Error measures the average distance between the detected edges and the actual object boundaries in the image. It's a more direct measure of how well the detected edges correspond to the true edges in terms of spatial accuracy.

These are other metrics can be used :

- **Edge Pixel Count.** Is used to tell how many edge points the algorithm detected. The number of white pixels (value = 255) in the binary edge map. For openCV we have `cv2.countNonZero()` function. **Where:**

Too few edge pixels may refer to an important features might be missed, Too many could mean noise or over-detection.

- **Edge Continuity.** Is employed to see how connected or smooth the edge lines are. While a good edge detectors produce long, continuous edges instead of broken fragments.

Where:

Longer contours means more continuous edges and More short contours indicate fragmented and/or noisy edges.

There's no direct **OpenCV function** to perform this operation but we can simply estimate this is by:

- 1- Use `cv2.findContours()` to find connected edge segments (contours);
- 2- Then calculate the average length of these contours.

2. What to do?

Part 1:

1. Explore which are the best cases to use each of these metrics?
2. Perform edge detection method (Sobel, Prewitt, Canny, Laplacian) on the following images, compare the detected edges with the ground-truth image for images: `3063.jpg`, `3063_boundary.png`, *Write down your Observation.*
 - You can use files in folders `TestImages`, and `groudTruth` after converting the `.mat` file to an image, to explore more, using Python with `scipy.io`.
1. How closely do the detected edges match the ground-truth edges?

Part 2:

1. Apply edge detection on a set of test images (noisy image, object image, photo with textures).
2. Compare the detected edges to the ground truth using all the mentioned metrics.
3. Calculate and visualize these metrics using `sklearn.metrics`. **What do you notice?**

4. For ROC Curve & AUC, how does each edge detection method perform as the threshold varies? Which method has the highest AUC?
5. Which edge detection method minimizes the Boundary Displacement Error and provides the best alignment with true boundaries?
6. How does the FOM score differ across edge detectors? Which detector offers the best performance based on the FOM?

T1.2.3 - Quantitative metrics against ground truth

```
def read_ground_truth_image():
    gt = read_gray("3063_boundary.png")
    if gt is not None:
        return np.where(gt > 127, 255, 0).astype(np.uint8)

    if loadmat is not None:
        mat_path = find_file("3063.mat")
        if mat_path is not None:
            mat = loadmat(str(mat_path))
            # pick the first 2D numeric array from the MAT file
            for key, value in mat.items():
                if key.startswith("__"):
                    continue
                arr = np.asarray(value)
                if arr.ndim == 2:
                    arr = normalize_to_uint8(arr)
                    return np.where(arr > 127, 255,
0).astype(np.uint8)
            print("Ground-truth for 3063 was not found.")
            return None

test_img = read_gray("3063.jpg")
gt_img = read_ground_truth_image()

if test_img is not None and gt_img is not None:
    gt_img = cv2.resize(gt_img, (test_img.shape[1],
test_img.shape[0]), interpolation=cv2.INTER_NEAREST)

    edge_maps = apply_all_edge_detectors(test_img, canny_low=50,
canny_high=150, log_ksize=5)

    metric_table = []
    roc_data = {}

    for method, edge_img in edge_maps.items():
        pred_bin = ensure_binary(edge_img, thresh=60 if method !=
"Canny" else 1)
        metrics = compute_metrics(gt_img, pred_bin,
```

```

score_img=edge_img)
    metric_table.append([
        method,
        metrics.get("precision", np.nan),
        metrics.get("recall", np.nan),
        metrics.get("f1", np.nan),
        metrics.get("auc", np.nan),
        metrics.get("bde", np.nan),
        metrics.get("pratt_fom", np.nan),
        metrics.get("edge_pixel_count", np.nan),
        metrics.get("avg_contour_length", np.nan),
        metrics.get("num_contours", np.nan),
    ])
    if "fpr" in metrics and "tpr" in metrics:
        roc_data[method] = (metrics["fpr"], metrics["tpr"],
metrics["auc"])

    print("Metrics for 3063.jpg compared with ground-truth:\n")
    header = ["Method", "Precision", "Recall", "F1", "AUC", "BDE",
"Pratt FOM", "Edge Pixels", "Avg Contour Length", "Num Contours"]
    print("{:<10} {:>10} {:>10} {:>10} {:>10} {:>10} {:>12} {:>12}
{:>18} {:>14}".format(*header))
    for row in metric_table:
        print("{:<10} {:>10.4f} {:>10.4f} {:>10.4f} {:>10.4f}
{:>10.4f} {:>12.4f} {:>12.0f} {:>18.2f} {:>14.0f}".format(*row))

    show_grid(
        [test_img, gt_img] + [ensure_binary(edge_maps[m], 60 if m !=
"Canny" else 1) for m in ["Sobel", "Prewitt", "LoG", "Canny"]],
        titles=["Test Image", "Ground Truth", "Sobel", "Prewitt",
"LoG", "Canny"],
        cols=3,
        figsize=(14, 8)
    )

    if roc_data:
        plt.figure(figsize=(7, 6))
        for method, (fpr, tpr, roc_auc) in roc_data.items():
            plt.plot(fpr, tpr, label=f"{method} (AUC={roc_auc:.3f})")
        plt.plot([0, 1], [0, 1], linestyle='--')
        plt.xlabel("False Positive Rate")
        plt.ylabel("True Positive Rate")
        plt.title("ROC Curves for Edge Detectors on 3063")
        plt.legend()
        plt.grid(True, alpha=0.3)
        plt.show()
    else:
        print("Could not run quantitative evaluation because the test
image or ground-truth file is missing.")

```

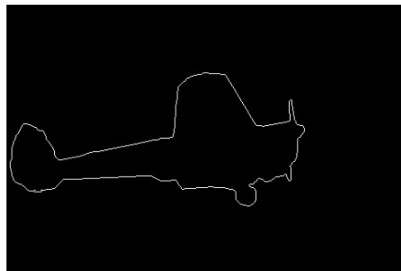
Metrics for 3063.jpg compared with ground-truth:

Method	Precision	Recall	F1	AUC	BDE
Pratt FOM	Edge Pixels	Avg Contour Length	Num	Contours	
Sobel	0.1638	0.4688	0.2428	0.8978	4.3848
	2839		27.69	121	
Prewitt	0.1622	0.4960	0.2444	0.9052	4.5267
	3034		29.30	117	
LoG	0.1297	0.2863	0.1786	0.9225	5.8589
	2189		16.08	176	
Canny	0.0509	0.2903	0.0866	0.6276	15.0466
	5663		65.70	165	

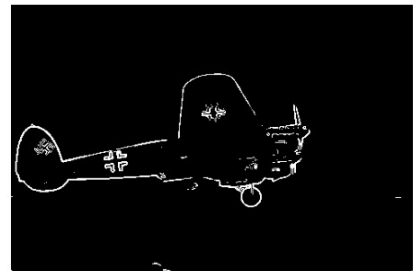
Test Image



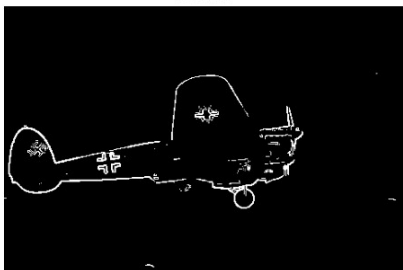
Ground Truth



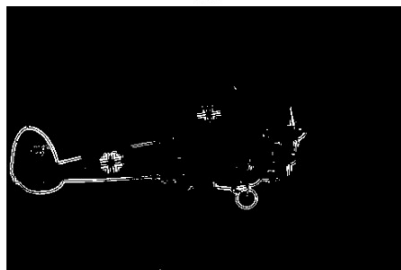
Sobel



Prewitt

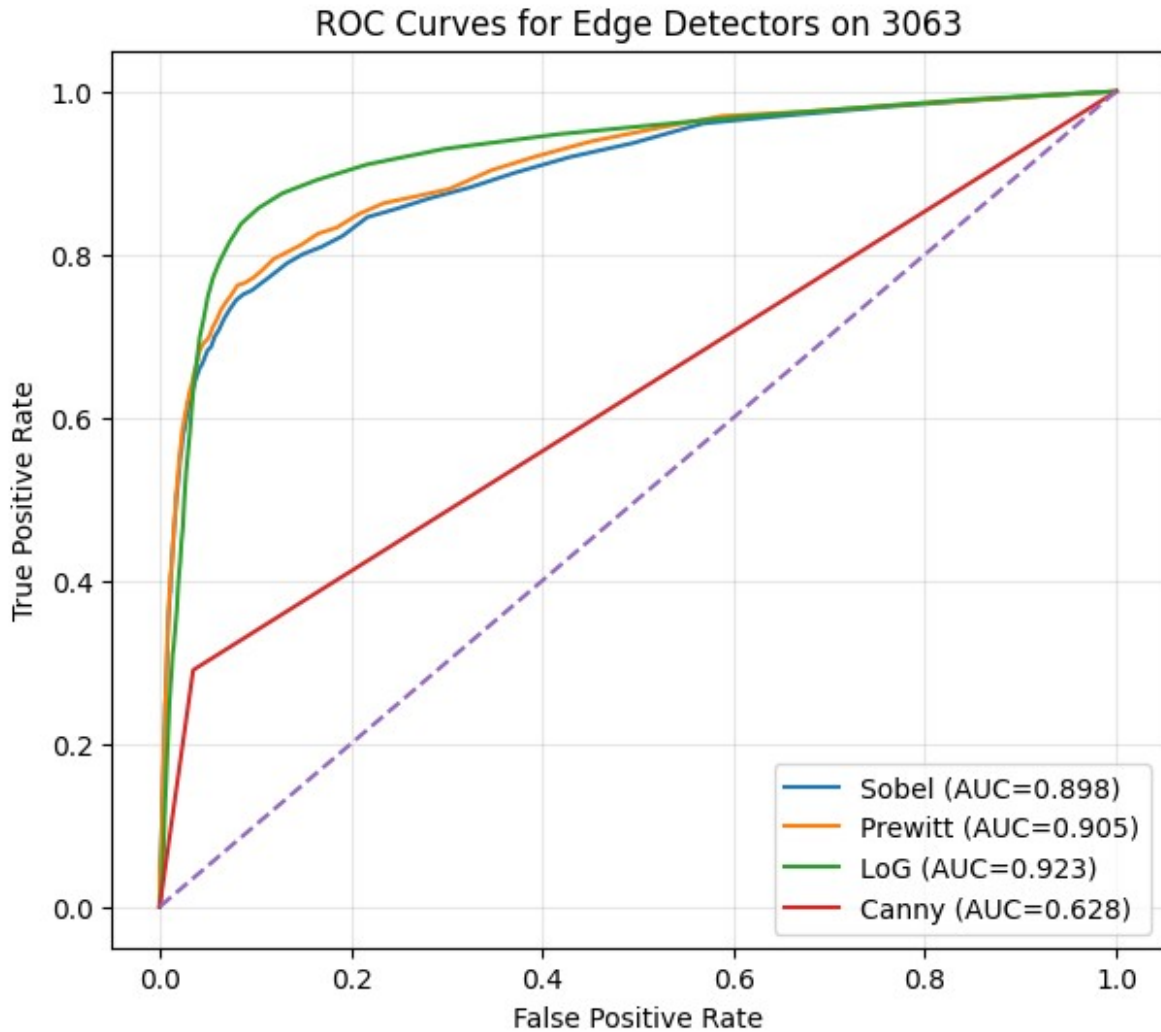


LoG



Canny





Add your analysis and comments here

The quantitative results show that **Prewitt and Sobel** provide the best overall balance for this image, with the highest **F1-scores** (about 0.244 for Prewitt and 0.243 for Sobel) and relatively close **precision** and **recall** values. This means they detect a reasonable number of true boundaries while keeping false detections lower than the other methods. Their **AUC** values are also strong, with **Prewitt = 0.905** and **Sobel = 0.898**, which indicates stable performance over different thresholds.

LoG achieves the **highest AUC (0.923)** and also the **best Pratt FOM (0.6008)**, which suggests that its detected edges align well with the ground-truth boundaries across threshold variations. However, its **F1-score is lower (0.179)** because it misses some true edges and has lower recall than Sobel and Prewitt. In other words, LoG seems good at boundary localization, but it is less complete in recovering all desired object edges.

Canny performs the worst in this experiment. It has the **lowest precision (0.0509)**, **lowest F1-score (0.0866)**, **lowest AUC (0.628)**, and the **largest BDE (15.0466)**, meaning its edges are poorly aligned with the true boundaries. This is also visible in the output images, where Canny detects many irrelevant edges from the sky and texture details instead of focusing mainly on the

aircraft contour. The very high number of edge pixels (**5663**) and long average contour length also confirm that it is over-detecting edges in this case.

From the **Boundary Displacement Error (BDE)** values, **Sobel (4.3848)** is the best, followed by **Prewitt (4.5267)**, while **LoG (5.8589)** is slightly worse and **Canny** is much worse. This tells us that Sobel gives the closest spatial match to the real object outline. So, if the goal is best boundary alignment, **Sobel** is the best choice here.

From the **ROC curve**, the method with the highest AUC is clearly **LoG**, followed by **Prewitt**, then **Sobel**, while **Canny** is far behind. This means LoG has the strongest threshold-independent separability between edge and non-edge pixels, even though its final binary output is not the best by F1-score. That difference is important: **AUC measures ranking quality across thresholds**, while **F1 measures performance at one chosen operating point**.

Overall, the results show that there is **no single best detector for every metric**.

- **Prewitt** gives the best overall detection balance by F1-score.
- **Sobel** gives the best boundary alignment by BDE.
- **LoG** gives the strongest ROC/AUC and the best Pratt FOM.
- **Canny** is the weakest on this image because it is too sensitive to texture and produces many false positives.

This also shows that detector performance depends strongly on the image content and parameter settings. For this airplane image, simpler gradient-based detectors like **Sobel** and **Prewitt** match the ground-truth boundary better than Canny, while **LoG** is strong when evaluated over varying thresholds.

Manipulation 2: Edge post-processing using morphological operations

Goals:

- Cleaning and enhancing edges to prepare for shape extraction.
- Understand the basics of morphological operations which are used in analyzing the form and shape details of image structures.

1. Introduction ##### 1.1 Structuring element **In morphological operations, the structuring element is the key component that enables these operations.**

- It is often referred to as a *Kernel or Mask* and it is defined by the `cv2.getStructuringElement` function (recommended), or can be manually defined using `NumPy`.

- This structuring element interacts with the image during morphological operations like *dilation, erosion, opening, and closing*.
- The choice of structuring element determines the effect of the operation on the image, which directly influences on the outcome of the operation and should be carefully chosen based on the desired effect.

There are different types of structuring elements in OpenCV:

1- *Rectangular structuring element* `cv2.MORPH_RECT`. It is the most commonly used shape for simple operations, where the neighborhood to be considered is a rectangular block.

2- *Cross-shaped structuring element* `cv2.MORPH_CROSS`. It is equivalent to a 4-neighborhood structuring element but can be of any size.

3- *Circular (elliptical) structuring element* `cv2.MORPH_ELLIPSE`. This shape is often used for smoother transformations, especially when trying to preserve round shapes or features in an image.

1.2 Morphological Operations

Sample Image

1- *Dilation* is a morphological operation that expands the boundaries of bright regions in an image. It makes objects thicker by adding pixels to the boundary of the foreground. It can be achieved using the OpenCV function `cv2.dilate(image, kernel)`.

Common Use: Connect broken parts of an object; Fill small holes in an image → It is frequently used in object detection and feature enhancement.

2- *Erosion* is the opposite of dilation. It shrinks the bright regions in an image, removing pixels from the boundaries of objects. It can be achieved using the OpenCV function `cv2.erode(image, kernel)`.

Common Use: Removing noise or isolating features, especially in binary images; Separating connected objects in an image.

3- *Opening* is a **combination of erosion followed by dilation**. It removes small objects and noise while maintaining the overall shape of larger structures. It can be achieved using the OpenCV function `cv2.MORPH_OPEN`.

Common Use: Removing noise, such as small dots or speckles in an image, especially in binary images used in image segmentation.

4- *Closing* is **the opposite of opening; it consists of dilation followed by erosion**. It is used to fill small holes in objects or gaps between them. It can be achieved using the OpenCV function `cv2.MORPH_CLOSE`.

Common Use: Reconstructing shapes or filling small holes in binary images, especially when dealing with Objects Of Interest (ROI) that are surrounded by noise.

5- *Morphological Gradient* represents *the difference between dilation and erosion*. It highlights edges and boundaries of objects in an image. It can be achieved using the OpenCV function `cv2.MORPH_GRADIENT`.

Common Use: Edge detection and feature extraction, as it provides a way to emphasize the boundary or contour of objects.

6- *Morphological Hats* are **derived from applying morphological operations (Opening and Closing) and subtracting the result from the original image**. There are two types: Top Hat and Black Hat.

- *Top Hat:* Difference between the original image and its opening. It emphasizes small bright regions or features. It can be achieved using the OpenCV function `cv2.MORPH_TOPHAT`.

Top Hat = Original Image - Opening

- *Black Hat:* Difference between the closing and the original image. It emphasizes small dark regions or features. It can be achieved using the OpenCV function `cv2.MORPH_BLACKHAT`.

Black Hat = Closing - Original Image

Common Use: **Top Hat** is often used to extract small bright features in an image, and **Black Hat** is used for detecting dark features. Both are useful for image enhancement and feature extraction in various imaging applications.

kernel is the structuring element (or mask) used for the operation. This is what determines the size and shape of the object that will be used for the operation. Therefore the size's choice of this kernel is important and crucial.

Task 1: Morphological operators on synthetic images

T1.1: Creating synthetic images

In this task, you will create three synthetic images using OpenCV. Each image will contain different shapes (square, rectangle, circle, and triangle). Follow the steps below to complete the task:

- Create black canvas for each image of size 200x200 pixels using NumPy.
- Draw shapes on the images, as follows:
 - Image 1: Draw a square in the middle of the canvas. OpenCV function "cv2.rectangle" can be used to fill the square with white color (255);
 - Image 2: Draw a rectangle (Color 255) and add a black circle inside the rectangle;
 - Image 3: Draw three different shapes, a circle, a square and a triangle (Color 255).

Sample Image

T2.1.1 - Create synthetic images

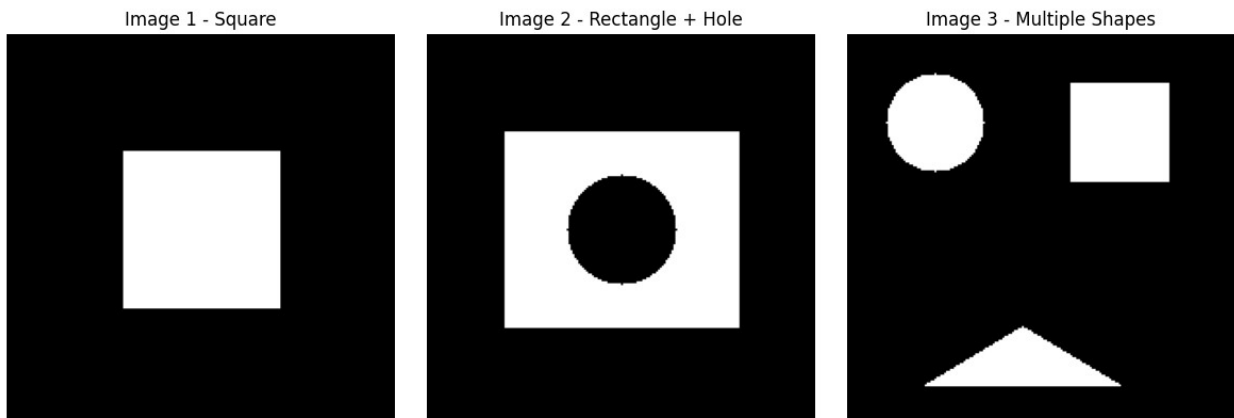
```
img1 = np.zeros((200, 200), dtype=np.uint8)
cv2.rectangle(img1, (60, 60), (140, 140), 255, -1)

img2 = np.zeros((200, 200), dtype=np.uint8)
cv2.rectangle(img2, (40, 50), (160, 150), 255, -1)
cv2.circle(img2, (100, 100), 28, 0, -1)

img3 = np.zeros((200, 200), dtype=np.uint8)
cv2.circle(img3, (45, 45), 25, 255, -1)
cv2.rectangle(img3, (115, 25), (165, 75), 255, -1)
triangle = np.array([[90, 150], [40, 180], [140, 180]],
dtype=np.int32)
cv2.fillPoly(img3, [triangle], 255)

synthetic_images = {
    "Image 1 - Square": img1,
    "Image 2 - Rectangle + Hole": img2,
    "Image 3 - Multiple Shapes": img3,
}

show_grid(list(synthetic_images.values()),
titles=list(synthetic_images.keys()), cols=3, figsize=(12, 4))
```



T1.2: Morphological operations on synthetic images

- 1- Perform all morphological operations on the synthetic images.
- 2- Run each operation for 1, 5, 500, 1000 or more iterations, with different kernel sizes: 3x3 and 9x9. **(Try different number of iterations)**
- 3- Display the original image and the resulting image after performing each operation.

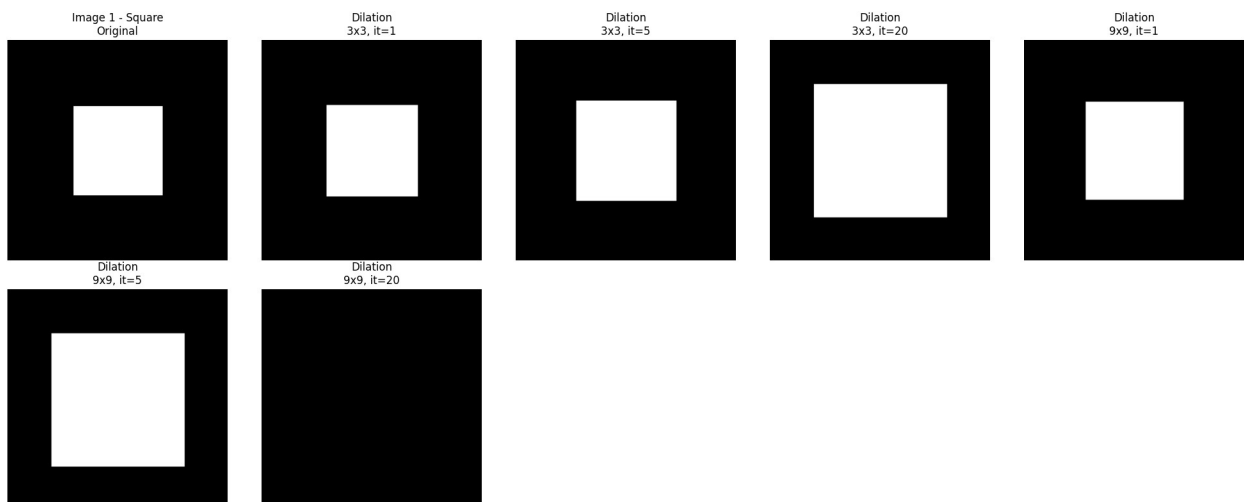
4- Discuss findings based on the changes observed with different iterations and kernel sizes.
What do you notice?

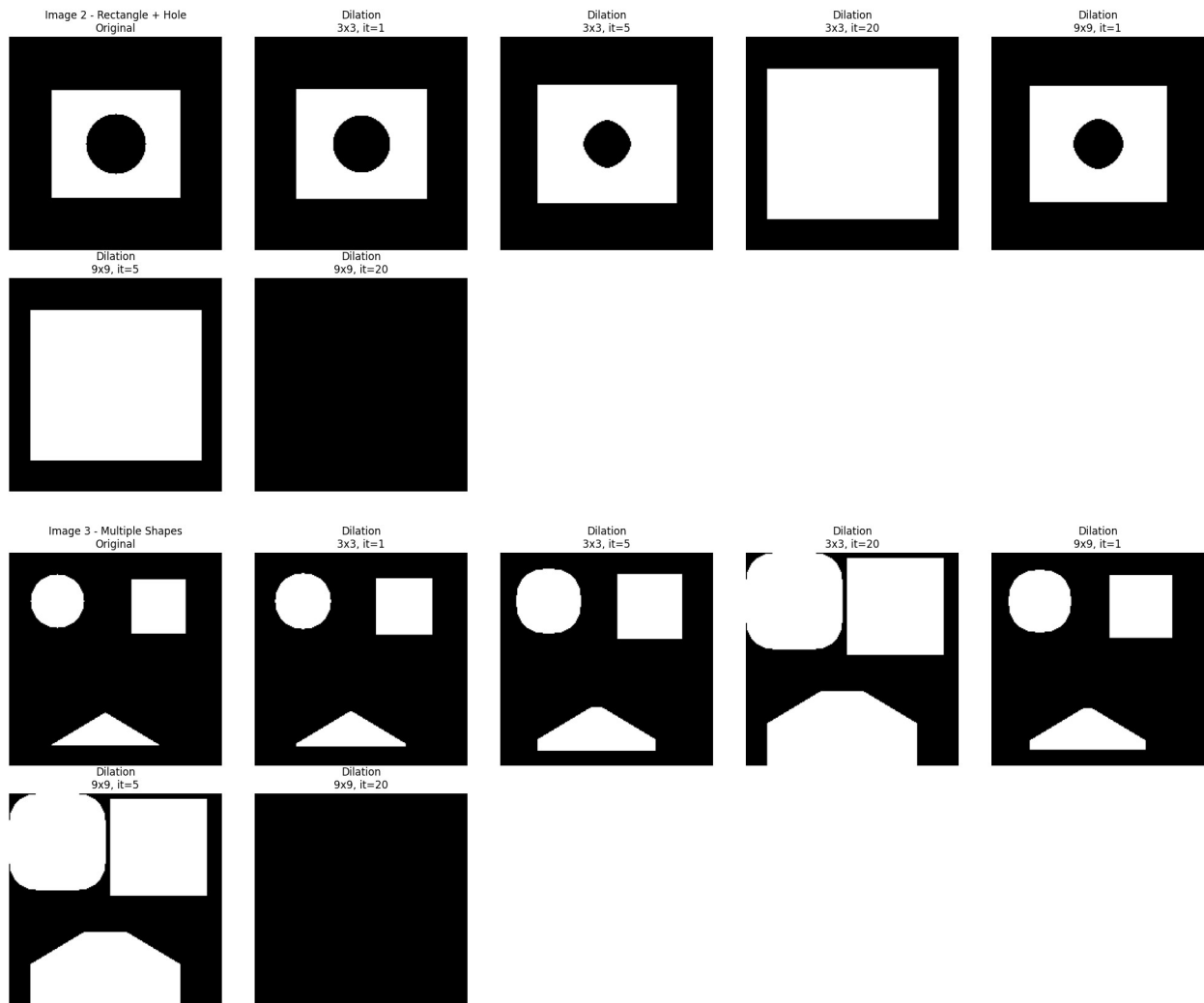
T1.2.1: Dilation

T1.2.1 - Dilation

```
kernels = {
    "3x3": np.ones((3, 3), dtype=np.uint8),
    "9x9": np.ones((9, 9), dtype=np.uint8)
}
iterations_to_try = [1, 5, 20]

for img_name, img in synthetic_images.items():
    results = [img]
    titles = [f"{img_name}\nOriginal"]
    for kernel_name, kernel in kernels.items():
        for it in iterations_to_try:
            out = cv2.morphologyEx(img, None, kernel, iterations=it)
            if "Dilation" in ["Opening", "Closing", "Gradient", "Top Hat"] else
            None
                if out is None:
                    if "Dilation" == "Dilation":
                        out = cv2.dilate(img, kernel, iterations=it)
                    elif "Dilation" == "Erosion":
                        out = cv2.erode(img, kernel, iterations=it)
                results.append(out)
                titles.append(f"Dilation\n{kernel_name}, it={it}")
            show_grid(results, titles=titles, cols=5, figsize=(20, 8))
```





Add your analysis and comments here

- Dilation expands the white regions, causing objects to grow in size and fill small gaps or holes.
- Increasing the number of iterations or kernel size (e.g., 9×9) makes the expansion more aggressive.
- Thin structures become thicker, and nearby objects may merge together.
- It is useful for connecting broken parts of objects and emphasizing shapes.

T1.2.2: Erosion

T1.2.2 - Erosion

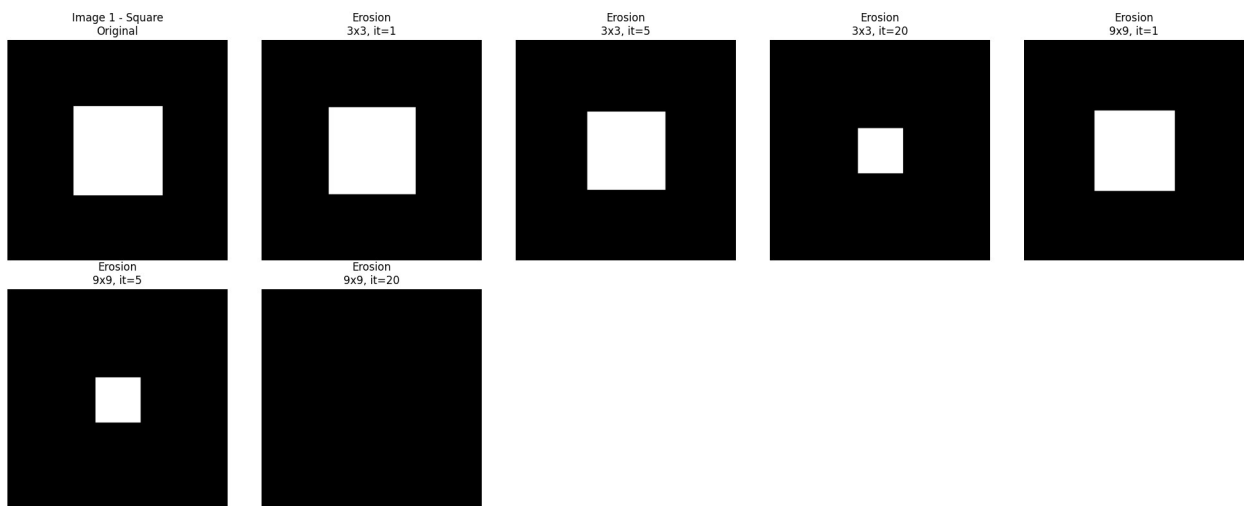
```
kernels = {
    "3x3": np.ones((3, 3), dtype=np.uint8),
```

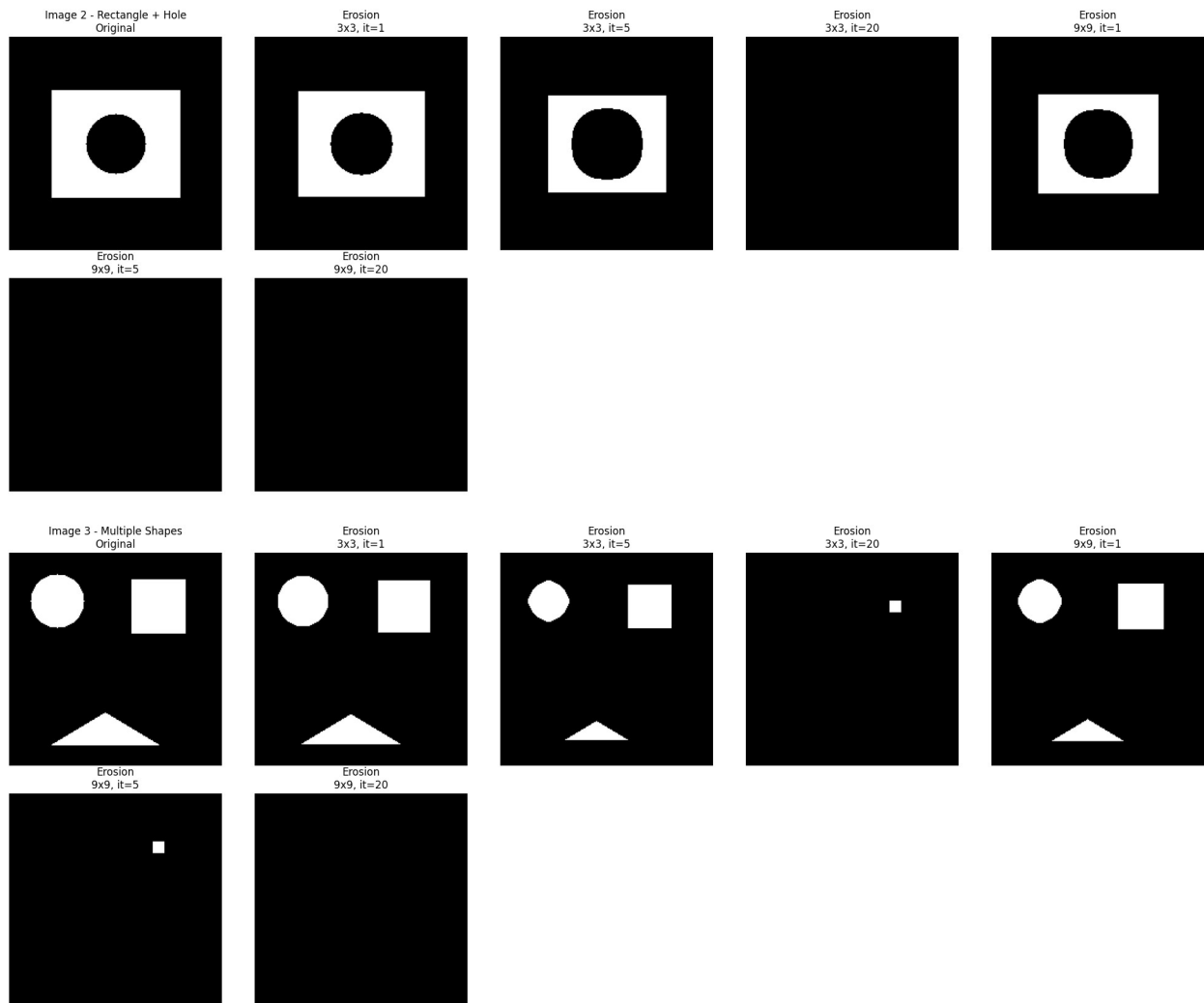
```

    "9x9": np.ones((9, 9), dtype=np.uint8)
}
iterations_to_try = [1, 5, 20]

for img_name, img in synthetic_images.items():
    results = [img]
    titles = [f"{img_name}\nOriginal"]
    for kernel_name, kernel in kernels.items():
        for it in iterations_to_try:
            out = cv2.morphologyEx(img, None, kernel, iterations=it)
            if "Erosion" in ["Opening", "Closing", "Gradient", "Top Hat"] else
            None
                if out is None:
                    if "Erosion" == "Dilation":
                        out = cv2.dilate(img, kernel, iterations=it)
                    elif "Erosion" == "Erosion":
                        out = cv2.erode(img, kernel, iterations=it)
                results.append(out)
                titles.append(f"Erosion\n{kernel_name}, it={it}")
show_grid(results, titles=titles, cols=5, figsize=(20, 8))

```





Add your analysis and comments here

- Erosion shrinks white regions, reducing object size and removing small details or noise.
- With more iterations or larger kernels, objects can become very thin or even disappear completely.
- It helps eliminate small unwanted regions and detach connected components.
- However, excessive erosion may destroy important shape information.

T1.2.3: Opening

T1.2.3 - Opening

```

kernels = {
    "3x3": np.ones((3, 3), dtype=np.uint8),
    "9x9": np.ones((9, 9), dtype=np.uint8)
}

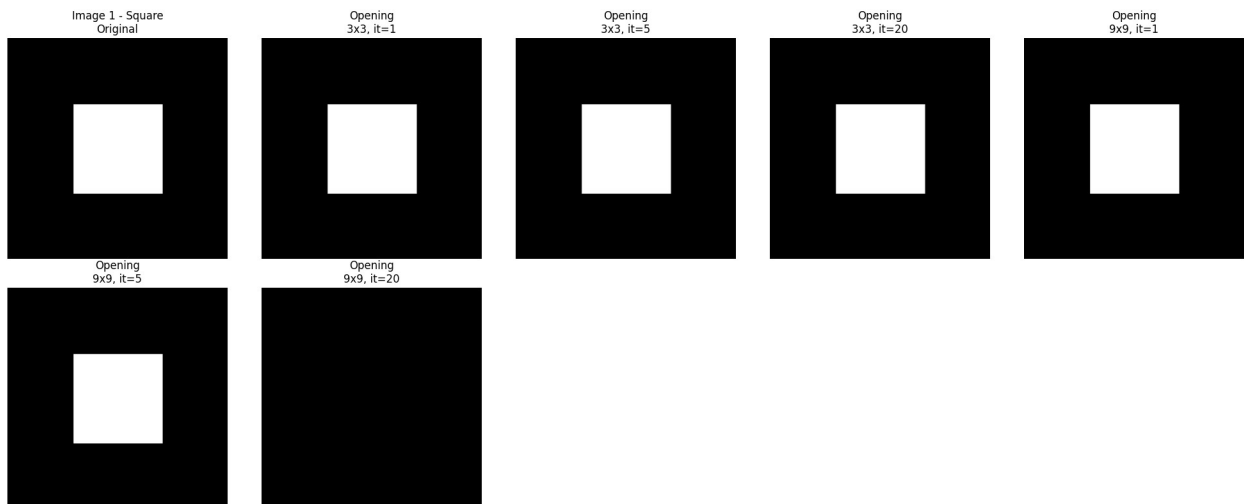
```

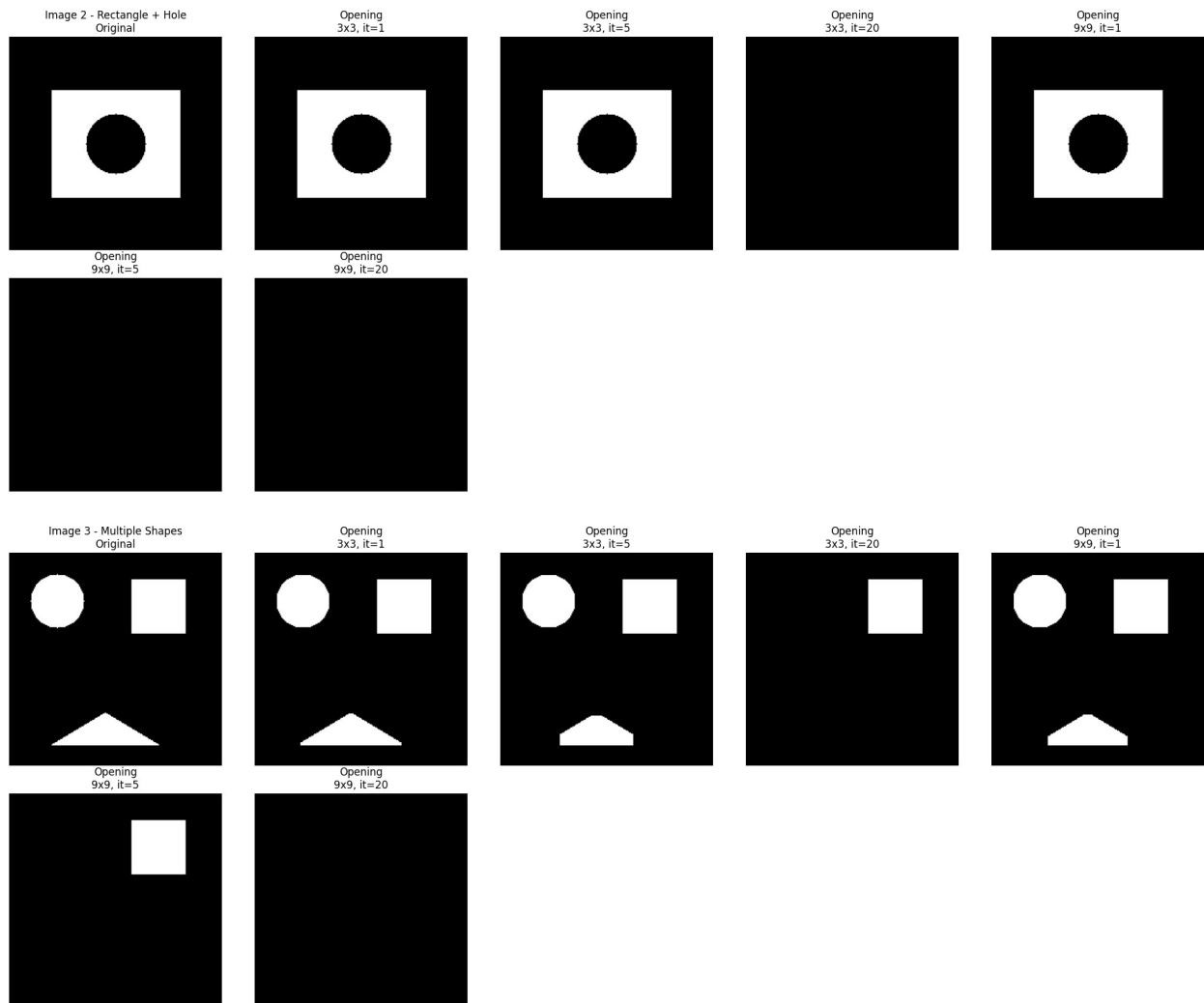
```

}
iterations_to_try = [1, 5, 20]

for img_name, img in synthetic_images.items():
    results = [img]
    titles = [f"{img_name}\nOriginal"]
    for kernel_name, kernel in kernels.items():
        for it in iterations_to_try:
            out = cv2.morphologyEx(img, cv2.MORPH_OPEN, kernel,
iterations=it) if "Opening" in ["Opening", "Closing", "Gradient", "Top
Hat"] else None
            if out is None:
                if "Opening" == "Dilation":
                    out = cv2.dilate(img, kernel, iterations=it)
                elif "Opening" == "Erosion":
                    out = cv2.erode(img, kernel, iterations=it)
            results.append(out)
            titles.append(f"Opening\n{kernel_name}, it={it}")
show_grid(results, titles=titles, cols=5, figsize=(20, 8))

```





Add your analysis and comments here

- Opening (erosion followed by dilation) removes small noise while preserving the main shape of objects.
- It smooths object boundaries and eliminates small protrusions or isolated pixels.
- Larger kernels and more iterations remove more details and simplify shapes.
- It is useful for cleaning noisy images without significantly enlarging objects.

T1.2.4: Closing

T1.2.4 - Closing

```

kernels = {
    "3x3": np.ones((3, 3), dtype=np.uint8),
    "9x9": np.ones((9, 9), dtype=np.uint8)
}

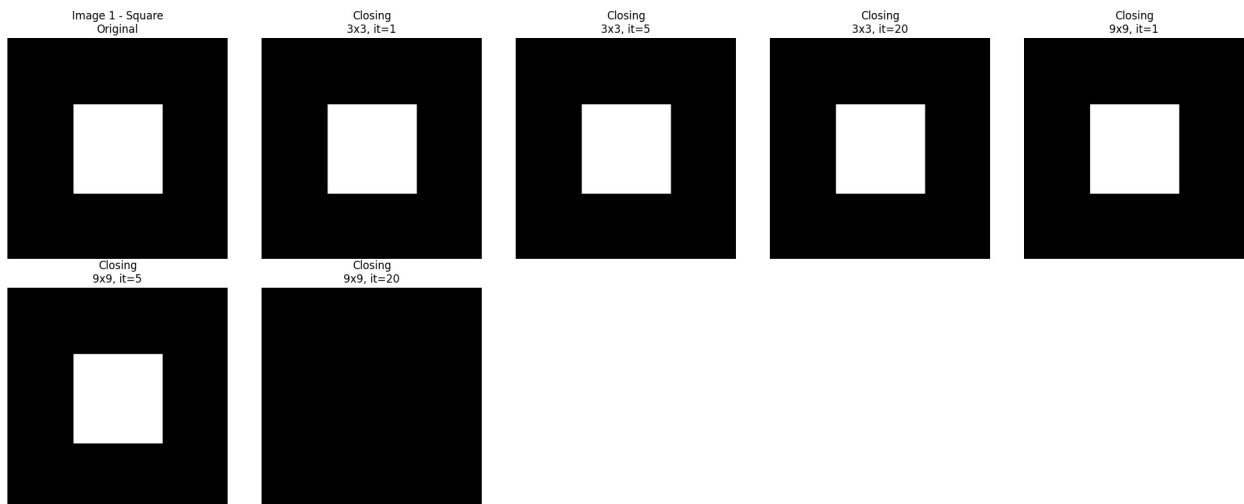
```

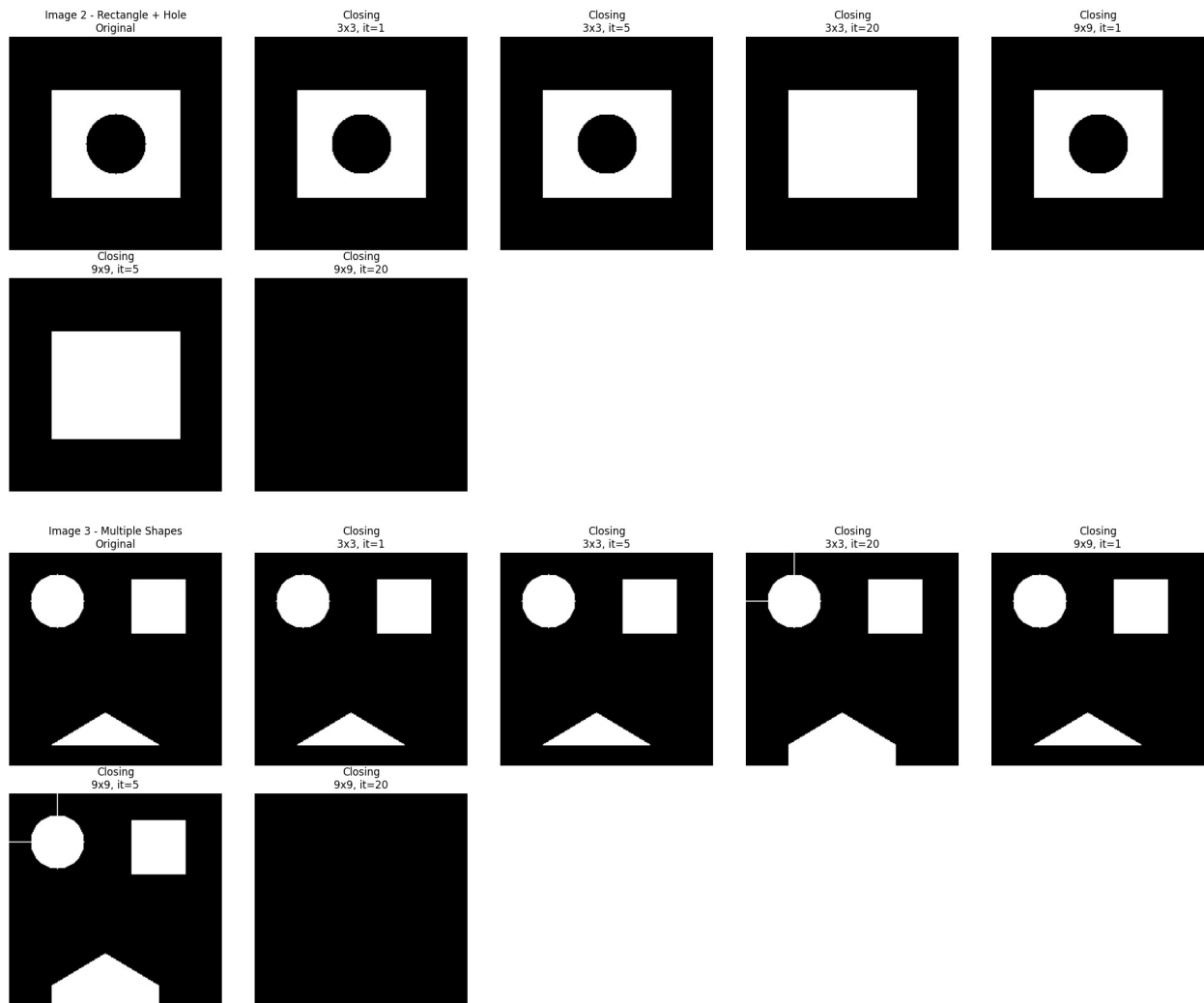
```

}
iterations_to_try = [1, 5, 20]

for img_name, img in synthetic_images.items():
    results = [img]
    titles = [f"{img_name}\nOriginal"]
    for kernel_name, kernel in kernels.items():
        for it in iterations_to_try:
            out = cv2.morphologyEx(img, cv2.MORPH_CLOSE, kernel,
iterations=it) if "Closing" in ["Opening", "Closing", "Gradient", "Top
Hat"] else None
            if out is None:
                if "Closing" == "Dilation":
                    out = cv2.dilate(img, kernel, iterations=it)
                elif "Closing" == "Erosion":
                    out = cv2.erode(img, kernel, iterations=it)
            results.append(out)
            titles.append(f"Closing\n{kernel_name}, it={it}")
show_grid(results, titles=titles, cols=5, figsize=(20, 8))

```





Add your analysis and comments here

- Closing (dilation followed by erosion) fills small holes and gaps inside objects.
- It connects nearby regions and smooths boundaries while maintaining overall shape.
- With larger kernels, gaps are filled more effectively, but fine details may be lost.
- It is useful for completing broken shapes and improving object continuity.

T1.2.5: Morphological Gradient

T1.2.5 - Morphological Gradient

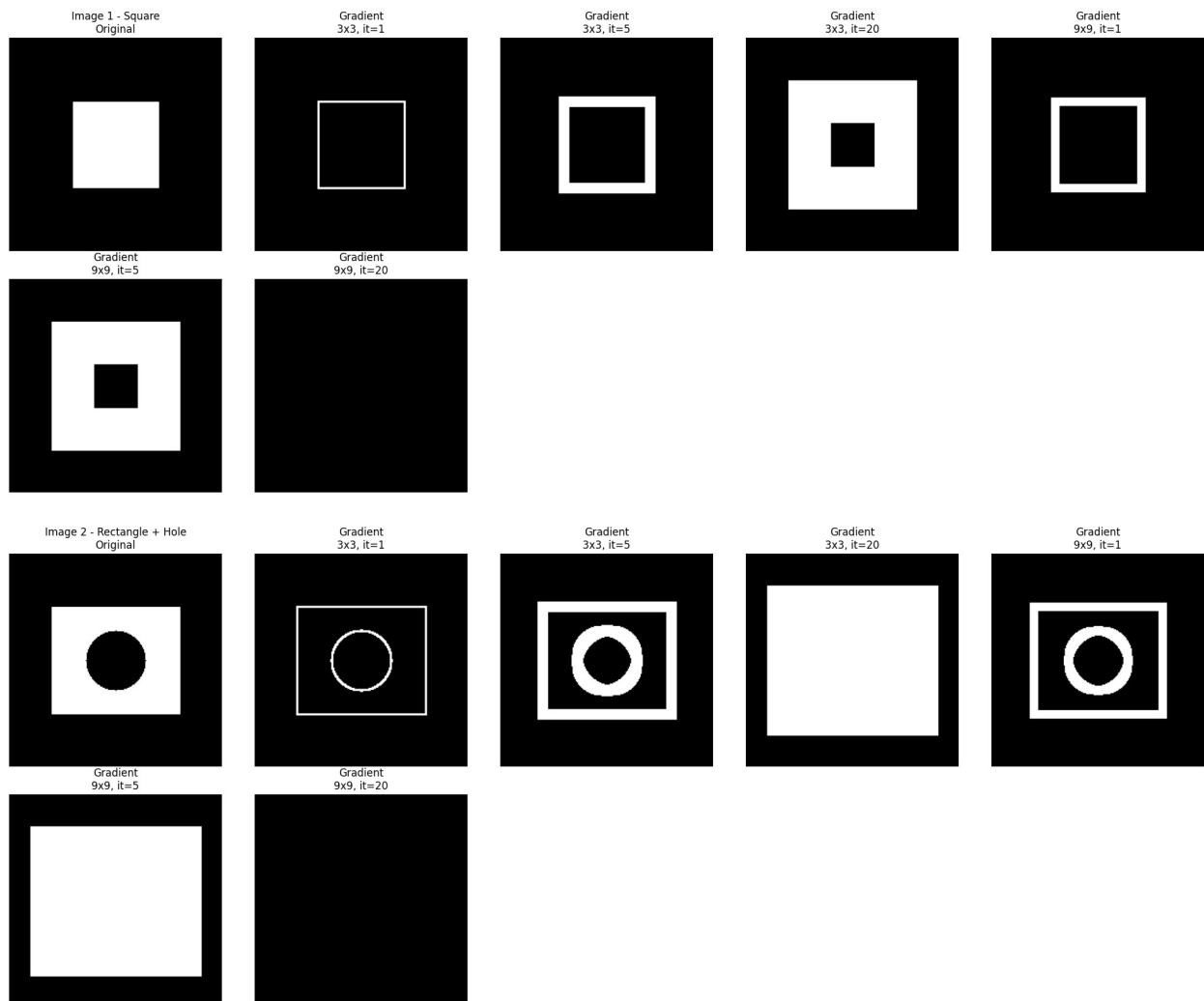
```
kernels = {
    "3x3": np.ones((3, 3), dtype=np.uint8),
    "9x9": np.ones((9, 9), dtype=np.uint8)
}
```

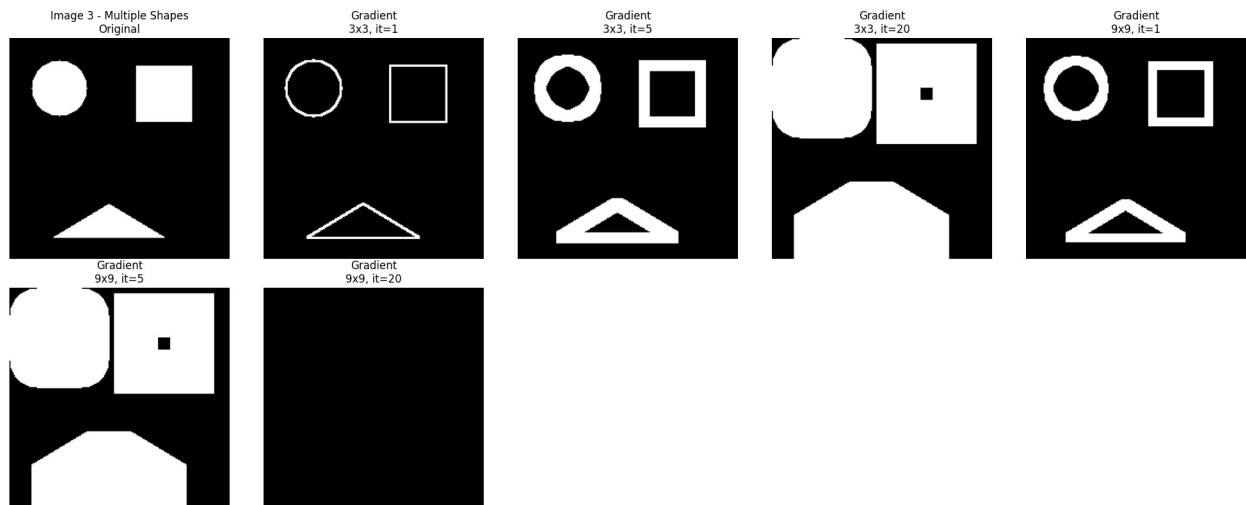
```

iterations_to_try = [1, 5, 20]

for img_name, img in synthetic_images.items():
    results = [img]
    titles = [f"{img_name}\nOriginal"]
    for kernel_name, kernel in kernels.items():
        for it in iterations_to_try:
            out = cv2.morphologyEx(img, cv2.MORPH_GRADIENT, kernel,
iterations=it) if "Gradient" in ["Opening", "Closing", "Gradient",
"Top Hat"] else None
            if out is None:
                if "Gradient" == "Dilation":
                    out = cv2.dilate(img, kernel, iterations=it)
                elif "Gradient" == "Erosion":
                    out = cv2.erode(img, kernel, iterations=it)
            results.append(out)
            titles.append(f"Gradient\n{kernel_name}, it={it}")
    show_grid(results, titles=titles, cols=5, figsize=(20, 8))

```





Add your analysis and comments here

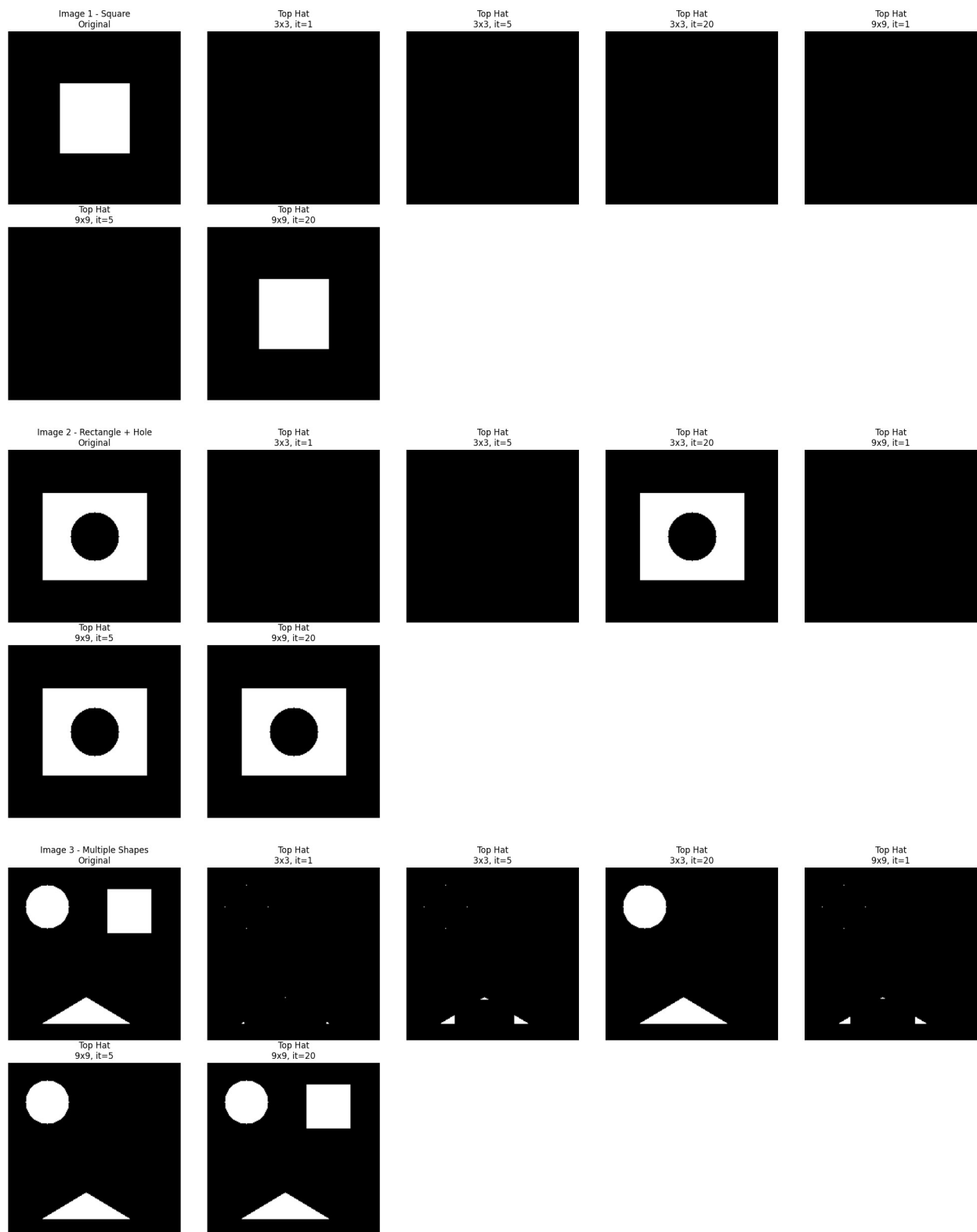
- The morphological gradient extracts object boundaries by computing the difference between dilation and erosion.
- It highlights edges clearly, producing an outline around shapes.
- Larger kernels make edges thicker and less precise.
- It is useful for edge detection and boundary visualization.

T1.2.6: Morphological Hat

```
# T1.2.6 - Morphological Hat (Top Hat)

kernels = {
    "3x3": np.ones((3, 3), dtype=np.uint8),
    "9x9": np.ones((9, 9), dtype=np.uint8)
}
iterations_to_try = [1, 5, 20]

for img_name, img in synthetic_images.items():
    results = [img]
    titles = [f"{img_name}\nOriginal"]
    for kernel_name, kernel in kernels.items():
        for it in iterations_to_try:
            out = cv2.morphologyEx(img, cv2.MORPH_TOPHAT, kernel,
iterations=it)
            results.append(out)
            titles.append(f"Top Hat\n{kernel_name}, it={it}")
    show_grid(results, titles=titles, cols=5, figsize=(20, 8))
```

Add your analysis and comments here

- Top Hat extracts small bright elements by subtracting the opened image from the original.
- It highlights fine details and small objects that are removed during opening.
- With larger kernels, only very small features remain visible.
- It is useful for detecting small bright structures on dark backgrounds.

Task 2: Morphological operators & natural (real) images

T2.1 Dilation and erosion in medical imaging

Use dilation to enhance or connect tumor boundaries and erosion to remove noise in medical scans `Brain.png` for the image `Lung.png` display the following result.

Sample Image

What to do?

Display the original image and the resulted one side by side. What is the best combination of number of iterations and kernel size? What do you notice?

```
# T2.1 - Dilation and erosion in medical imaging

medical_files = ["Brain.png", "Lung.png"]
kernels = {"3x3": np.ones((3, 3), np.uint8), "5x5": np.ones((5, 5),
np.uint8), "9x9": np.ones((9, 9), np.uint8)}
iterations_list = [1, 2, 5]

for fname in medical_files:
    img = read_gray(fname)
    if img is None:
        continue

    # Use Otsu threshold so morphology works on a binary image
    _, binary = cv2.threshold(img, 0, 255, cv2.THRESH_BINARY +
cv2.THRESH_OTSU)

    results = [img, binary]
    titles = [f"{fname}\nOriginal", "Binary"]

    best_score = None
    best_summary = None

    for kernel_name, kernel in kernels.items():
        for it in iterations_list:
```

```

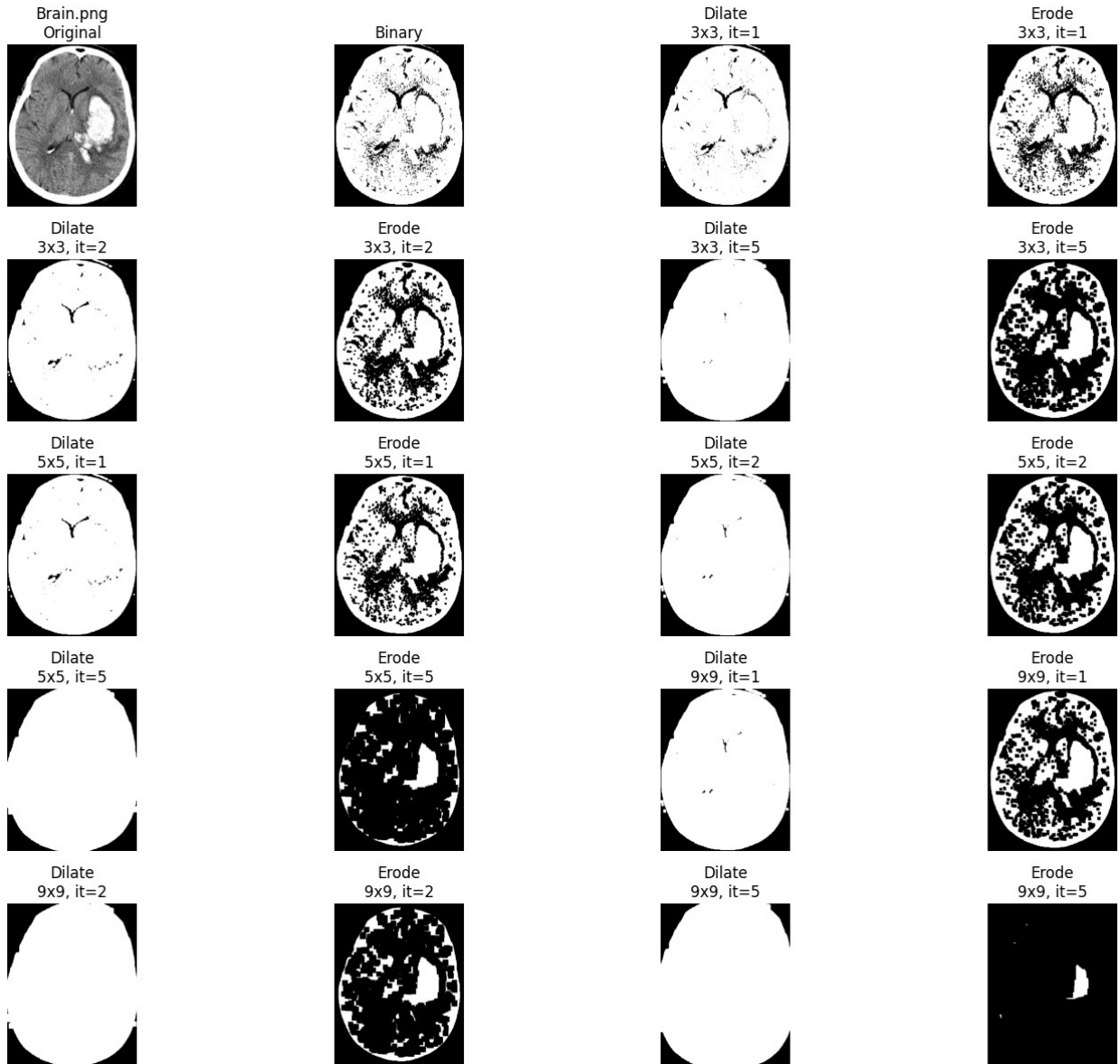
        dil = cv2.dilate(binary, kernel, iterations=it)
        ero = cv2.erode(binary, kernel, iterations=it)

        results.extend([dil, ero])
        titles.extend([f"Dilate\n{kernel_name}, it={it}", f"Erode\
n{kernel_name}, it={it}"])

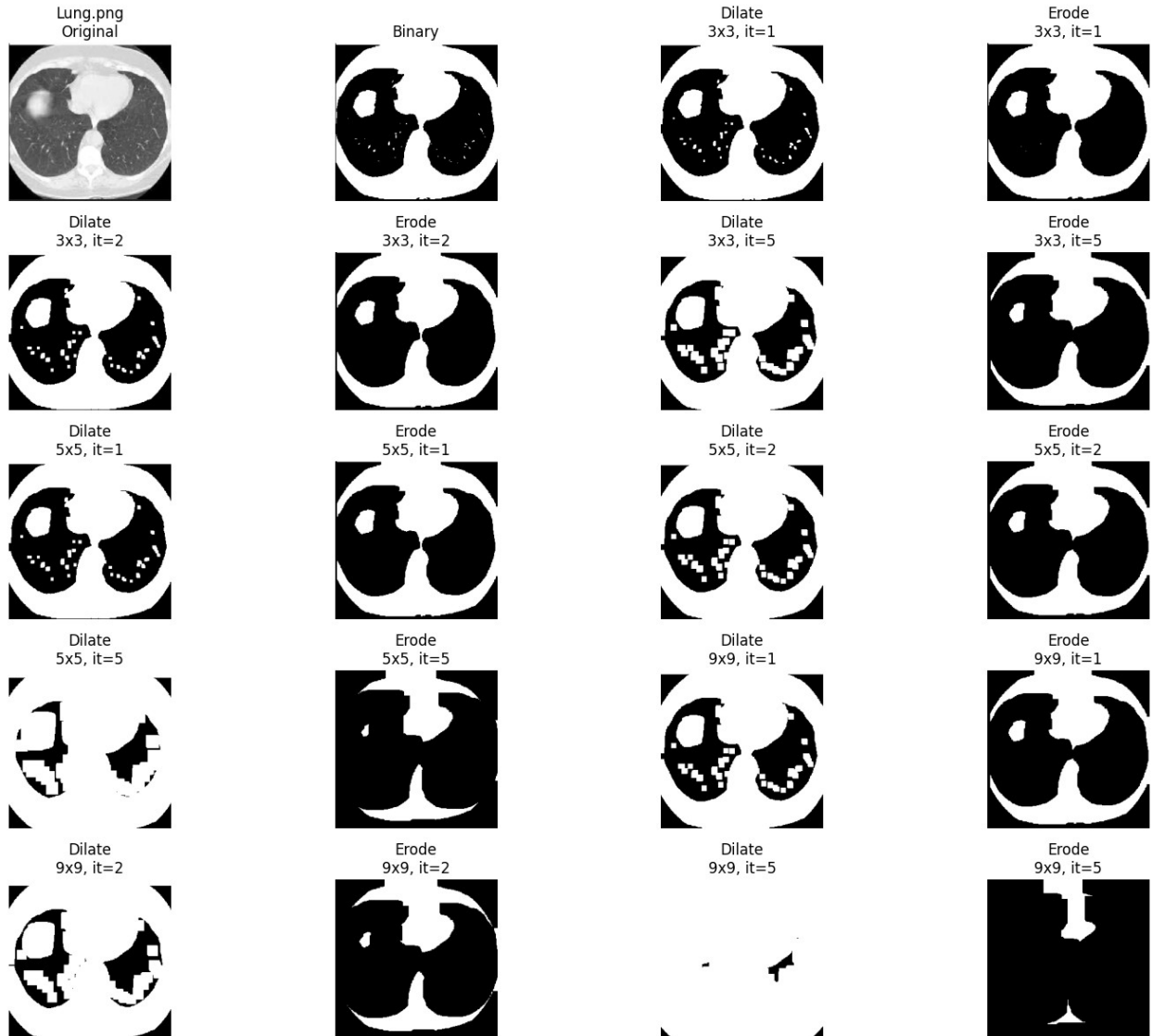
        score = cv2.countNonZero(dil) - cv2.countNonZero(ero)
        if best_score is None or abs(score) < abs(best_score):
            best_score = score
            best_summary = (kernel_name, it)

    print(f"{fname}: reasonable setting to start with -> kernel
{best_summary[0]}, iterations {best_summary[1]}")
    show_grid(results, titles=titles, cols=4, figsize=(16, 12))
Brain.png: reasonable setting to start with -> kernel 3x3, iterations
1

```



Lung.png: reasonable setting to start with -> kernel 3x3, iterations 1



Add your analysis and comments here

Dilation helps enhance and connect important structures such as tumor boundaries, making them more visible and continuous, especially in the lung image.

Erosion is effective in removing small noise and unwanted details, but if applied too strongly, it can remove important anatomical structures.

A moderate kernel size (e.g., 3x3 or 5x5) with a small number of iterations provides the best balance between noise removal and structure preservation.

Larger kernels or many iterations tend to over-expand (dilation) or over-shrink (erosion) regions, leading to loss of fine details or distortion of shapes.

Overall, combining mild erosion followed by dilation gives cleaner and more accurate segmentation results while preserving medically relevant features.

T2.2. Opening for Noise Removal in Document Images

Use opening to remove specks or dots in scanned documents for handwritten text images `HandWriting.jpeg`. Where you will clean scanned images by removing small noise artifacts while preserving the text. Connect the disconnected lines as well.

What to do?

Display the original Image and the resulted one side by side. What is the best combination of Number of Iterations and Kernel Size?

What do you notice?

T2.2 - Opening for noise removal in document images

```
img = read_gray("HandWriting.jpeg")
if img is not None:
    _, binary_inv = cv2.threshold(img, 0, 255, cv2.THRESH_BINARY_INV +
cv2.THRESH_OTSU)

    kernels = {
        "3x3": np.ones((3, 3), np.uint8),
        "5x5": np.ones((5, 5), np.uint8),
        "line(1x3)": cv2.getStructuringElement(cv2.MORPH_RECT, (3,
1)),
        "line(3x1)": cv2.getStructuringElement(cv2.MORPH_RECT, (1,
3)),
    }

    results = [img, binary_inv]
    titles = ["Original", "Binary Inverted"]

    for kernel_name, kernel in kernels.items():
        opened = cv2.morphologyEx(binary_inv, cv2.MORPH_OPEN, kernel,
iterations=1)
        connected = cv2.morphologyEx(opened, cv2.MORPH_CLOSE, kernel,
iterations=1)
        results.extend([opened, connected])
        titles.extend([f"Opening\n{kernel_name}", f"Open + Close\
n{kernel_name}"])

    show_grid(results, titles=titles, cols=4, figsize=(16, 10))
```



Add your analysis and comments here

Analysis of Opening for Noise Removal

Configuration	Noise Removal	Text Preservation	Connectivity of Letters	Observation
Opening (5×5)	Very strong	Poor	Broken	Removes most noise but also removes parts of thin handwriting
Opening + Closing (5×5)	Strong	Moderate	Improved	Restores some structure but still loses fine details
Opening (line 1×3)	Moderate	Good	Preserved horizontally	Keeps most text while removing small

Configuration	Noise Removal	Text Preservation	Connectivity of Letters	Observation
Opening + Closing (1×3)	Moderate	Very good	Well connected	noise aligned vertically Best balance, preserves text and improves continuity
Opening (line 3×1)	Moderate	Good	Preserved vertically	Better for vertical strokes, removes noise without heavy loss
Opening + Closing (3×1)	Moderate	Very good	Well connected	Best for vertical text continuity and noise cleaning

Conclusion

Using **line-shaped kernels (1×3 or 3×1)** gives better results than square kernels for handwritten text.

The best combination is **Opening followed by Closing with a small line kernel**, as it removes noise while preserving and reconnecting text.

T2.3. Removing small gap filling in satellite imaging

Use closing to fill small gaps or missing pixels between roads, buildings or other large structures in satellite images, to improve object connectivity in satellite images for analysis

`AerialPhotoMap.jpg`. So, we will have a results like:

Sample Image

What to do?

Display the original Image and the resulted one side by side. What is the best combination of Number of Iterations and Kernel Size?

What do you notice?


```

# T2.3 - Closing to fill small gaps in satellite imagery

img = read_gray("AerialPhotoMap.jpg")
if img is None:
    img = read_gray("AerialPhotoMap.jpeg")

if img is not None:
    _, binary = cv2.threshold(img, 0, 255, cv2.THRESH_BINARY +
cv2.THRESH_OTSU)

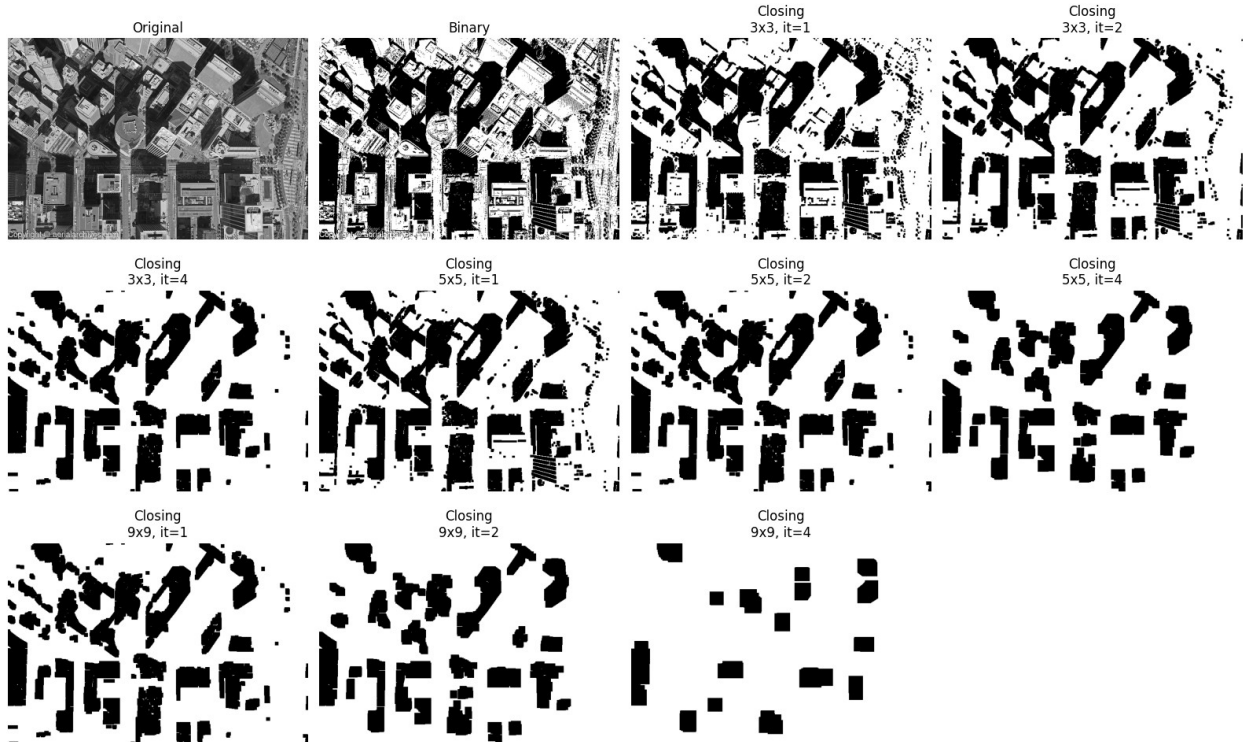
    kernels = {
        "3x3": np.ones((3, 3), np.uint8),
        "5x5": np.ones((5, 5), np.uint8),
        "9x9": np.ones((9, 9), np.uint8),
    }

    results = [img, binary]
    titles = ["Original", "Binary"]

    for kernel_name, kernel in kernels.items():
        for it in [1, 2, 4]:
            closed = cv2.morphologyEx(binary, cv2.MORPH_CLOSE, kernel,
iterations=it)
            results.append(closed)
            titles.append(f"Closing\n{kernel_name}, it={it}")

    show_grid(results, titles=titles, cols=4, figsize=(16, 10))

```



Add your analysis and comments here

Closing helps fill small gaps between structures like buildings and roads, improving connectivity in the satellite image, especially with moderate iterations (e.g., 1–2).

Using large kernels or too many iterations (e.g., 9×9 , $it=4$) over-smooths the image and merges separate objects, causing loss of important details.

T2.4. Morphological gradient for edge detection in object detection

Analyze the quality control in manufacturing, use morphological gradient to highlight edges of objects in industrial inspection, such as identifying defective production items using Part2.jpg, Part3.jpg, Part5.jpg, Part6.jpg. Such as the following illustration:

Sample Image

What to do?

Display the original Image and the resulted one side by side. What is the best combination of Number of Iterations and Kernel Size?

What do you notice?

Image Types: Industrial images, scanned parts.

T2.4 - Morphological gradient for object edges in industrial

inspection

```
part_files = ["Part2.jpg", "Part3.jpg", "Part5.jpg", "Part6.jpg"]

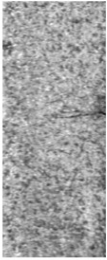
for fname in part_files:
    img = read_gray(fname)
    if img is None:
        continue

    results = [img]
    titles = [f"{fname}\nOriginal"]

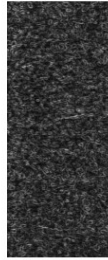
    for kernel_name, kernel in {
        "3x3": np.ones((3, 3), np.uint8),
        "5x5": np.ones((5, 5), np.uint8),
        "9x9": np.ones((9, 9), np.uint8)
    }.items():
        for it in [1, 2, 3]:
            grad = cv2.morphologyEx(img, cv2.MORPH_GRADIENT, kernel,
iterations=it)
            results.append(grad)
            titles.append(f"Gradient\n{kernel_name}, it={it}")

    show_grid(results, titles=titles, cols=4, figsize=(16, 10))
```

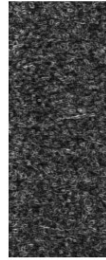
Part2.jpg
Original



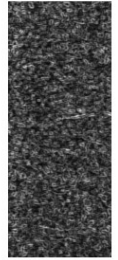
Gradient
3x3, it=1



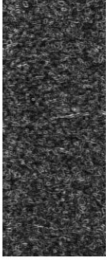
Gradient
3x3, it=2



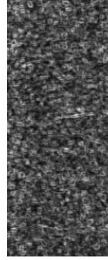
Gradient
3x3, it=3



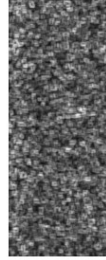
Gradient
5x5, it=1



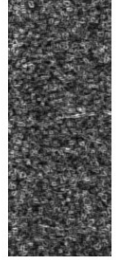
Gradient
5x5, it=2



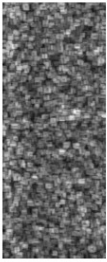
Gradient
5x5, it=3



Gradient
9x9, it=1



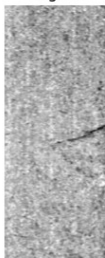
Gradient
9x9, it=2



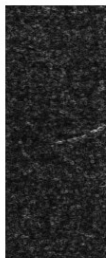
Gradient
9x9, it=3



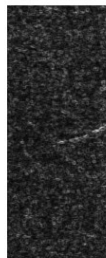
Part3.jpg
Original



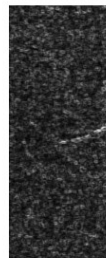
Gradient
3x3, it=1



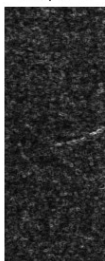
Gradient
3x3, it=2



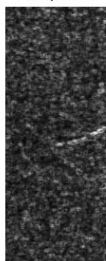
Gradient
3x3, it=3



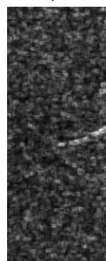
Gradient
5x5, it=1



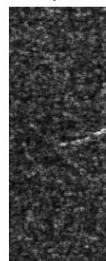
Gradient
5x5, it=2



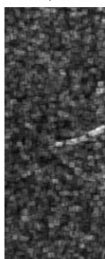
Gradient
5x5, it=3



Gradient
9x9, it=1



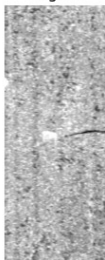
Gradient
9x9, it=2



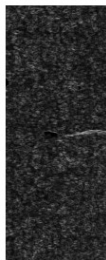
Gradient
9x9, it=3



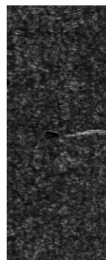
Part5.jpg
Original



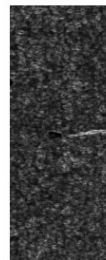
Gradient
3x3, it=1



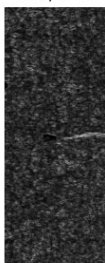
Gradient
3x3, it=2



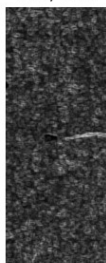
Gradient
3x3, it=3



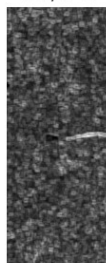
Gradient
5x5, it=1



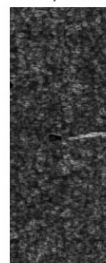
Gradient
5x5, it=2



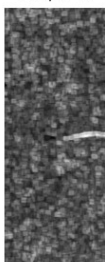
Gradient
5x5, it=3



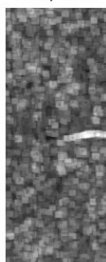
Gradient
9x9, it=1

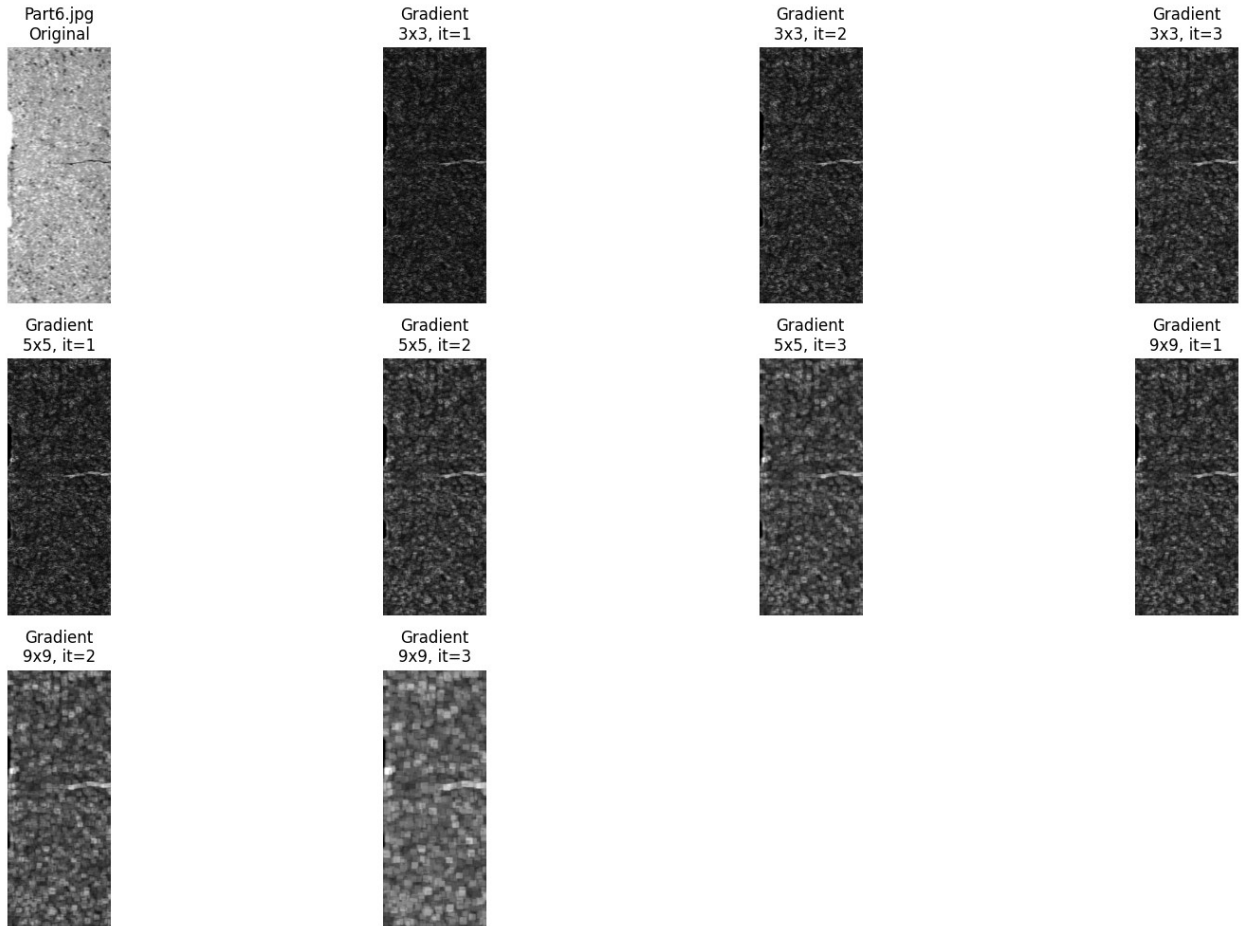


Gradient
9x9, it=2



Gradient
9x9, it=3





Add your analysis and comments here

The morphological gradient clearly highlights object edges and defects, making it useful for detecting cracks or boundaries in industrial images.

A moderate kernel (e.g., 3×3 or 5×5) with few iterations gives sharp edges, while larger kernels (9×9) or more iterations make edges thicker and reduce precision.

T2.5. Top Hat and Black Hat for feature extraction

Improve image analysis by detecting small features and differentiating between light and dark regions, by the mean of Top Hat to detect small features (e.g., buildings, roads) and Black Hat to extract dark regions like shadows in remote sensing images `AerialPhotoMap.jpg`, `Aerial3.png`, `Aerial4.png`.

What to do?

Display the original Image and the resulted one side by side. What is the best combination of Number of Iterations and Kernel Size?

What do you notice?

T2.5 - Top Hat and Black Hat for feature extraction

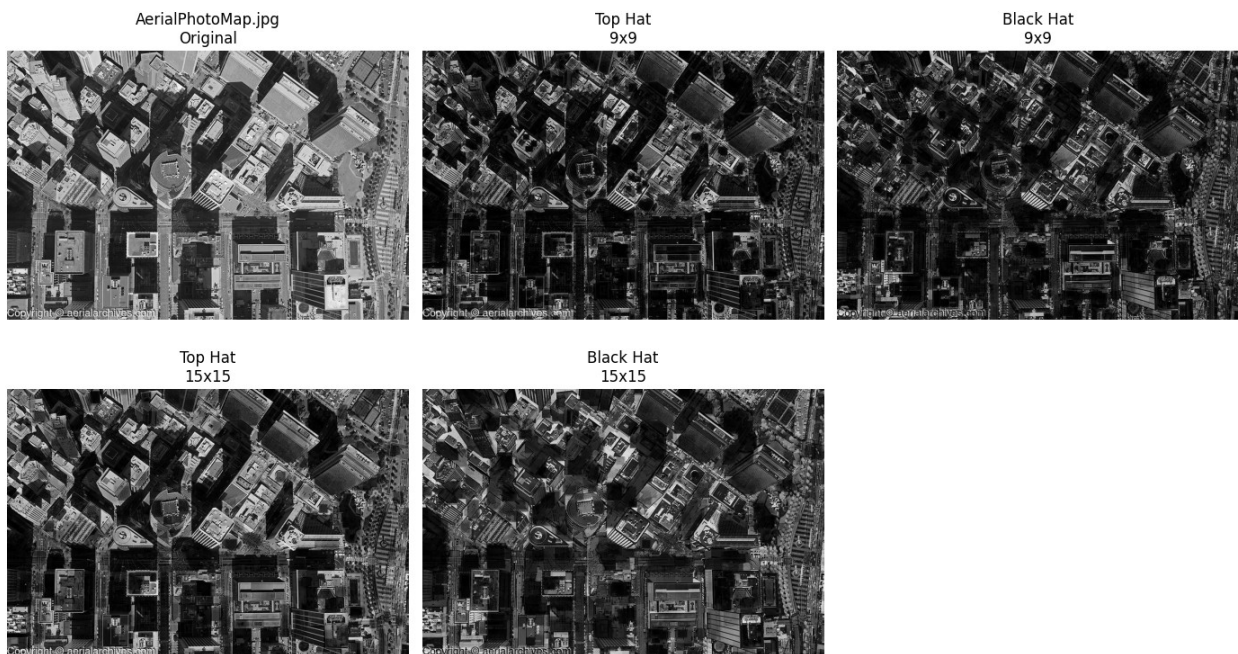
```
remote_files = ["AerialPhotoMap.jpg", "Aerial3.png", "Aerial4.png"]

for fname in remote_files:
    img = read_gray(fname)
    if img is None:
        continue

    results = [img]
    titles = [f"{fname}\nOriginal"]

    for kernel_name, kernel in {
        "9x9": np.ones((9, 9), np.uint8),
        "15x15": np.ones((15, 15), np.uint8)
    }.items():
        top_hat = cv2.morphologyEx(img, cv2.MORPH_TOPHAT, kernel,
iterations=1)
        black_hat = cv2.morphologyEx(img, cv2.MORPH_BLACKHAT, kernel,
iterations=1)
        results.extend([top_hat, black_hat])
        titles.extend([f"Top Hat\n{kernel_name}", f"Black Hat\
n{kernel_name}"])

    show_grid(results, titles=titles, cols=3, figsize=(14, 8))
```



Aerial3.png
Original



Top Hat
9x9



Black Hat
9x9



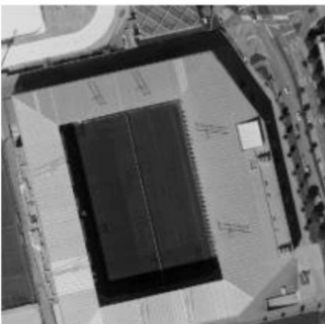
Top Hat
15x15



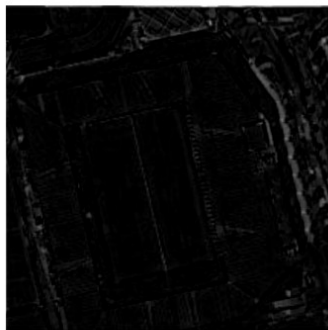
Black Hat
15x15



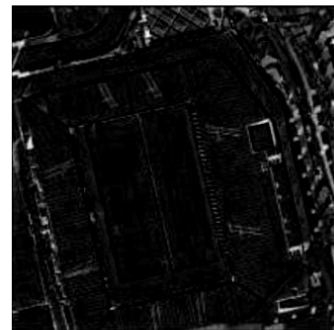
Aerial4.png
Original



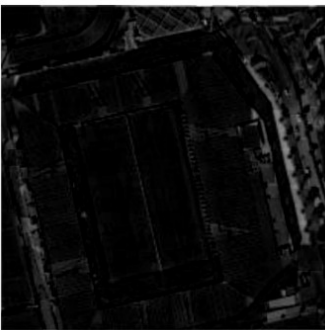
Top Hat
9x9



Black Hat
9x9



Top Hat
15x15



Black Hat
15x15



Add your analysis and comments here

Top Hat highlights small bright features such as buildings or roads, especially with smaller kernels, while larger kernels (15×15) emphasize only larger structures.

Black Hat extracts dark regions like shadows and gaps, making it useful for detecting darker features, but large kernels may over-smooth and lose fine details.